

Rapport

Säkerhet i molnutveckling – NET25S_SM

av
Mikael Cratz

Innehåll

1.	Inledning och uppgift	4
2.	Beskrivning av lösningen	4
2.1	Översiktlig arkitektur	4
2.2	Projektstruktur	5
3.	Genomförande – från lokal utveckling till Azure	6
3.1	Utveckling av API:t	6
3.2	Microsoft Entra ID	6
3.3	Azure-miljön och publicering av API:t	8
3.4	Azure SQL och databasanslutning	9
3.5	Hemlighetshantering och konfiguration	11
3.6	Autentisering och framtagning av access token	11
3.7	Test av rollbaserad åtkomst	14
4.	Problem som uppstod och hur de löstes	15
4.1	Databasstruktur saknades i Azure SQL	16
4.2	Autentisering och behörighet med Microsoft Entra ID	16
4.3	API-dokumentation och 404-fel i Azure	16
4.4	Problem med Azure-konto och autentisering mot Azure SQL	17
4.5	Skillnader mellan utvecklings- och produktionsmiljö	17
5.	Fördjupad säkerhetsanalys	18
5.1	Least Privilege	18
5.2	Defense in Depth	19
5.3	Secure by Design	19
5.4	Secure by Default	20
5.5	Identitetshantering	21
5.6	Behörigheter	22
5.7	Nätverk	23
5.8	Kryptering	24
5.9	Hemlighetshantering	24
5.10	Loggning	25
5.11	Övervakning	25
6.	Identifierade säkerhetsrisker	26
6.1	Offentlig åtkomst till Azure SQL	26
6.2	Komprometterad användare med rollen Resident	26
6.3	Komprometterad PropertyManager	27
6.4	Otillräcklig loggning och övervakning	27
6.5	Exponering av hemligheter	27
7.	Föreslagen säkerhetsförbättring	28
7.1	Azure Private Endpoint	28
7.2	Alternativ säkerhetsåtgärd	28

7.3 Motivering	28
8. Slutsats	29
9. Användning av AI som stöd	29
Bilaga A – Verktyg och metoder inom Identity and Access Management.....	30
A.1 Identity and Access Management (IAM)	30
A.2 Authentication, Authorization, Identity och Role	30
A.3 Least Privilege	31
A.4 Säkerhetsåtgärder vid utveckling av molnapplikationer	31
A.5 Hantering av känslig information	32
A.6 Defense in Depth	32
Bilaga B – Säkerhetsproblem och åtgärder	34
B.1 Känslig information i källkod	34
B.2 Vanliga säkerhetsproblem i molnapplikationer.....	34
B.3 OWASP Top 10	35
Bilaga C – Angriparens arbetssätt och verktyg	36
C.1 Angriparens arbetssätt	36
C.2 Vanliga verktyg och metoder.....	36
C.3 Betydelsen av flera säkerhetslager.....	37
Bilaga D – Säkerhetsterminologi	38
D.1 Authentication	38
D.2 Authorization	38
D.3 Least Privilege	38
D.4 Defense in Depth	38
D.5 Zero Trust	38
D.6 RBAC.....	38
D.7 Secret	38
D.8 Encryption at Rest	39
D.9 Encryption in Transit	39
D.10 Managed Identity.....	39
D.11 Secure by Design	39
D.12 Secure by Default	39

1. Inledning och uppgift

I denna uppgift har jag utvecklat och driftsatt ett webbaserat API för hantering av felanmälningar inom fastighetsskötsel. Lösningen är utvecklad med ASP.NET Core och Entity Framework Core och använder Microsoft Entra ID för autentisering och rollbaserad behörighetsstyrning. Data lagras i en SQL Server-databas som i produktionsmiljön körs i Azure SQL.

API:t har två huvudsakliga användarroller: **Resident** och **PropertyManager**. En boende ska kunna skapa felanmälningar och hämta sina egna tidigare anmälningar, medan en fastighetsskötare ska kunna hämta samtliga felanmälningar. Behörigheterna styrs genom roller i Microsoft Entra ID och kontrolleras av API:t innan en användare får tillgång till skyddade endpoints.

Lösningen har utvecklats lokalt och därefter publicerats till Azure App Service. I Azure används även Azure SQL för datalagring och Azure Key Vault för hantering av känslig konfiguration. Syftet har inte bara varit att få API:t att fungera, utan även att bygga lösningen med hänsyn till säkerhetsprinciper som Least Privilege, Defense in Depth och Secure by Design.

Lösningen genomfördes i en privat Azure-prenumeration. Detta möjliggjorde skapandet av flera separata testanvändare i Microsoft Entra ID och gjorde det möjligt att testa de olika behörighetsnivåerna med separata identiteter. På så sätt kunde exempelvis en användare med rollen Resident och en användare med rollen PropertyManager användas parallellt, istället för att samma användarkonto skulle behöva byta roll under testerna.

Rapporten beskriver därefter hur lösningen utvecklades, hur den konfigurerades och publicerades i Azure, vilka problem som uppstod under arbetet samt hur dessa löstes. Avslutningsvis analyseras lösningen utifrån de säkerhetsprinciper och säkerhetsområden som anges i uppgiften och ett antal möjliga säkerhetsrisker och förbättringsåtgärder diskuteras.

2. Beskrivning av lösningen

Lösningen består av ett ASP.NET Core Web API för hantering av felanmälningar. API:t är uppbyggt enligt en lagerindeldad struktur där controllers hanterar HTTP-anrop, services innehåller applikationslogiken och repositories ansvarar för kommunikationen med databasen. Entity Framework Core används som ORM och SQL Server används som databasmotor.

Den centrala entiteten i lösningen är FaultReport. En felanmälan innehåller bland annat en beskrivning av felet, information om vem som skapade anmälan, när den skapades samt dess aktuella status.

API:t innehåller funktionalitet för två typer av användare. En användare med rollen Resident kan skapa en felanmälan och hämta sina egna felanmälningar. En användare med rollen PropertyManager kan hämta samtliga felanmälningar. På så sätt begränsas åtkomsten till information utifrån användarens arbetsuppgift. Autentiseringen hanteras med Microsoft Entra ID. När en användare loggar in erhåller klienten en JWT access token. Token skickas sedan med API-anropen och valideras av API:t. Utifrån informationen i token, bland annat användarens identitet och tilldelade roller, avgör API:t om användaren har rätt att utföra den efterfrågade operationen.

I produktionsmiljön körs API:t i Azure App Service och databasen finns i Azure SQL. Lösningen använder även Azure Key Vault för hantering av känslig information. Konfiguration som skiljer sig mellan utvecklingsmiljön och produktionsmiljön hanteras genom Azure App Service Configuration istället för att hårdkodas i applikationen.

2.1 Översiktlig arkitektur

Lösningen består av ett ASP.NET Core Web API som använder Microsoft Entra ID för autentisering och rollbaserad auktorisering. Någon separat frontend eller annan klientapplikation har inte utvecklats inom projektet. API:t har istället testats och anropats med Scalar och Postman.

Det huvudsakliga användarflödet kan beskrivas enligt följande:

Användare → Microsoft Entra ID → JWT access token → ASP.NET Core API → Azure SQL

Användaren autentiseras av Microsoft Entra ID och får därefter en JWT access token. Token skickas med API-anropen och används av API:t för att identifiera användaren och kontrollera dennes roller. API:t kommunicerar sedan med Azure SQL för att lagra och hämta felanmälningar.

För åtkomst till känslig konfiguration används ett separat flöde där API:t identifierar sig mot Azure Key Vault genom sin System Assigned Managed Identity:

ASP.NET Core API → Managed Identity → Azure Key Vault

Denna identitet används bland annat för att läsa SMSApiKey, som används av SMSNotifier.

Arkitekturen består därmed av flera separata säkerhetslager där användarens identitet, API:ets identitet och åtkomsten till databasen och hemligheter hanteras separat.

2.2 Projektstruktur

API:t är uppdelat i flera delar för att separera ansvar mellan olika komponenter. Controllers ansvarar för API:ets endpoints, services för applikationslogik och repositories för dataåtkomst. Denna struktur gör lösningen lättare att underhålla och ger en tydligare separation mellan presentation, logik och datalager.

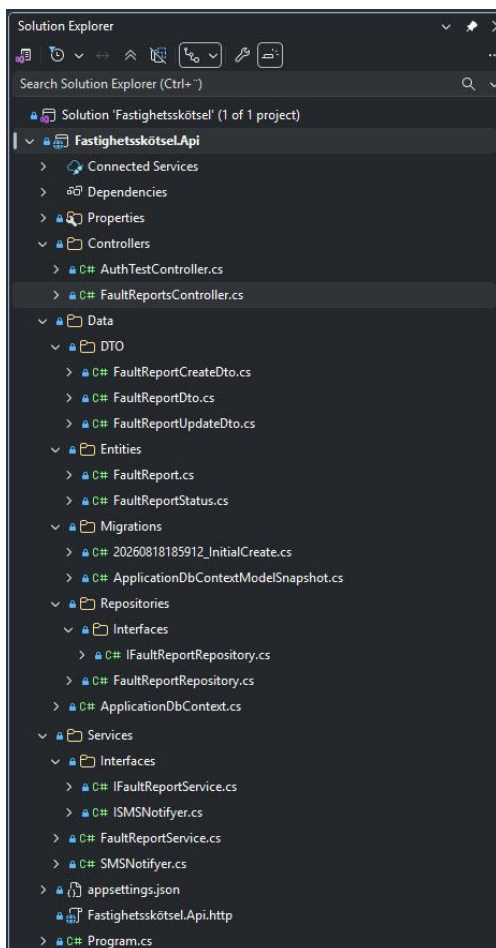


Bild 1 – Projektstruktur i Visual Studio

Bilden visar lösningens övergripande projektstruktur och de centrala delarna av API:t.

3. Genomförande – från lokal utveckling till Azure

3.1 Utveckling av API:t

Utvecklingen av lösningen påbörjades lokalt i Visual Studio. API:t utvecklades med ASP.NET Core och Entity Framework Core, med SQL Server som databas. Lösningen strukturerades i controllers, services och repositories för att separera ansvar mellan API-anrop, applikationslogik och dataåtkomst.

En central del av implementationen var att redan från början utforma API:t med separata behörighetsnivåer. De två rollerna Resident och PropertyManager representerar olika typer av användare och har därför olika åtkomst till API:ets endpoints.

För Resident begränsas åtkomsten till att skapa felanmälningar och hämta de felanmälningar som användaren själv har skapat. PropertyManager har däremot möjlighet att hämta samtliga felanmälningar. Denna uppdelning gör att användare inte behöver ha större behörigheter än vad deras uppgift kräver.

Behörighetskontrollerna implementerades direkt på API-nivå med Microsofts authentication- och authorizationfunktioner. På så sätt ligger kontrollen inte enbart i klientapplikationen, utan API:t verifierar själv användarens identitet och behörighet innan en skyddad operation genomförs.

Kodutdrag – controller och behörighetskontroll

```
[ApiController]
[Route("api/[controller]")]
public class FaultReportsController : ControllerBase

[HttpPost]
[Authorize(Roles = "Resident,PropertyManager")]
public async Task<ActionResult<FaultReportDto>> Create(FaultReportCreatedDto dto)

[HttpGet("mine")]
[Authorize(Roles = "Resident")]
public async Task<ActionResult<IEnumerable<FaultReportDto>>> GetMine()

[HttpGet]
[Authorize(Roles = "PropertyManager")]
public async Task<ActionResult<IEnumerable<FaultReportDto>>> GetAll()

[HttpGet("{id:int}")]
[Authorize(Roles = "PropertyManager")]
public async Task<ActionResult<FaultReportDto>> GetById(int id)

[HttpPatch("{id:int}")]
[Authorize(Roles = "PropertyManager")]
public async Task<ActionResult<FaultReportDto>> Update(int id, FaultReportUpdatedDto dto)

[HttpDelete("{id:int}")]
[Authorize(Roles = "PropertyManager")]
public async Task<IActionResult> Delete(int id)
```

3.2 Microsoft Entra ID

För att kunna autentisera användare och styra deras åtkomst till API:t registrerades API:t som en applikation i Microsoft Entra ID. App Registration fungerar som API:ets identitet i Entra ID och används av Microsofts identitetsplattform när access tokens utfärdas.

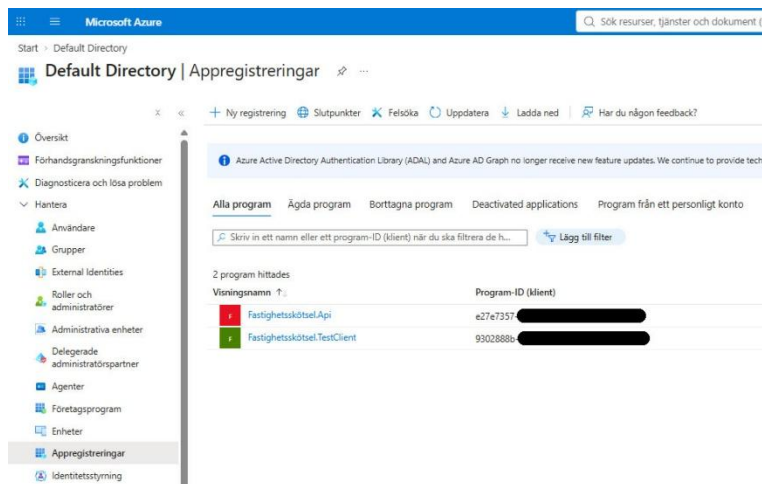


Bild 2 – App Registration

Bilden visar registreringen av *Fastighetsskötset.Api* i Microsoft Entra ID.

Fastighetsskötset.TestClient användes för att köra applikationen lokalt med *Scalar*.

Efter att API:t registrerats skapades två applikationsroller: *Resident* och *PropertyManager*. Roller används för att skilja mellan de olika typerna av användare och ger API:t möjlighet att fatta beslut om behörighet baserat på den roll som finns i användarens access token. *Resident* representerar en boende och har behörighet att skapa felanmälningar och hämta sina egna tidigare anmälningar. *PropertyManager* representerar fastighetsskötaren och har behörighet att hämta samtliga felanmälningar.

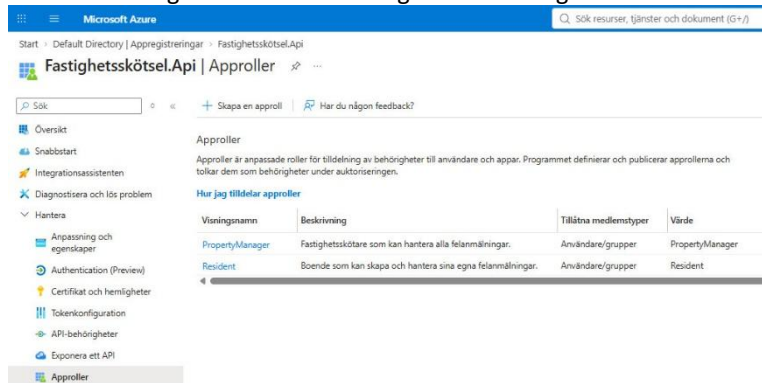


Bild 3 – App Roles

Bilden visar de två rollerna som definierats för API:t.

API:t konfigurerades även för att exponera ett scope kallat *access_as_user*. Scopet representerar den delegerade åtkomst som en klient kan begära till API:t när en användare är inloggad.

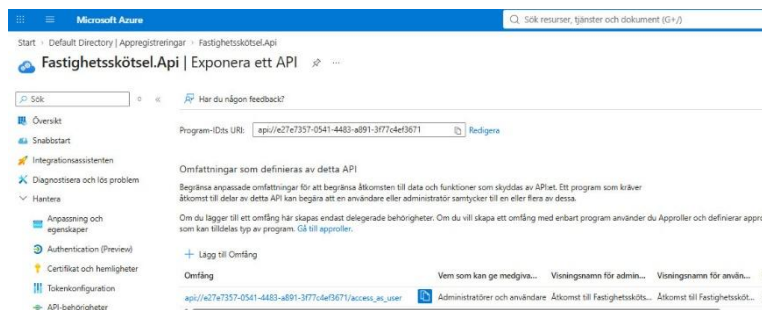


Bild 4 – Expose an API / access_as_user

Bilden visar det scope som API:t exponerar i Microsoft Entra ID.

För att kunna testa behörighetsmodellen skapades separata användare i Entra ID. Användarna tilldelades olika applikationsroller, där en användare exempelvis tilldelades *Resident* och en annan *PropertyManager*. Att använda separata testidentiteter gjorde det möjligt att testa behörighetsmodellen på ett mer realistiskt sätt. Samma användare behövde inte byta roll mellan olika tester, och det blev möjligt att verifiera både att en

användare får tillgång till de resurser som rollen ger rätt till och att användaren nekas åtkomst till resurser som kräver en annan roll.

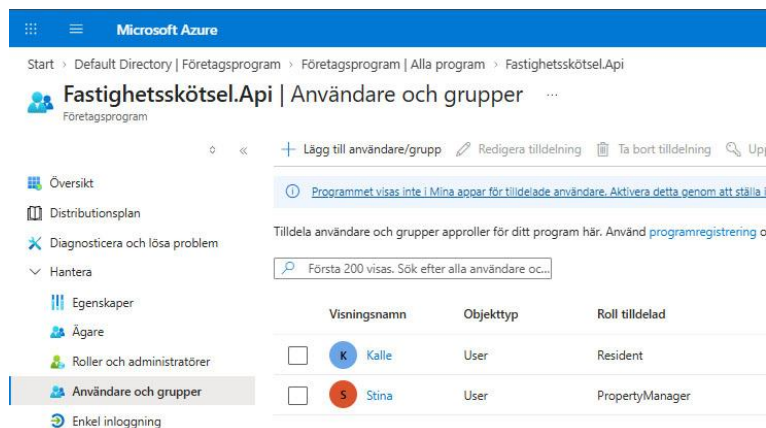


Bild 5 – Användare och rolltilldelning

Bilden visar testanvändarna och deras tilldelade roller.

3.3 Azure-miljön och publicering av API:t

När API:t fungerade i den lokala utvecklingsmiljön skapades den Azure-miljö som skulle användas för produktionsdrift. De resurser som hör till lösningen samlades i en gemensam Resource Group med namnet `rg_fastighet`. Genom att samla resurserna i samma Resource Group blev det enklare att få en överblick över lösningens olika delar och hantera dem som en sammanhängande miljö.

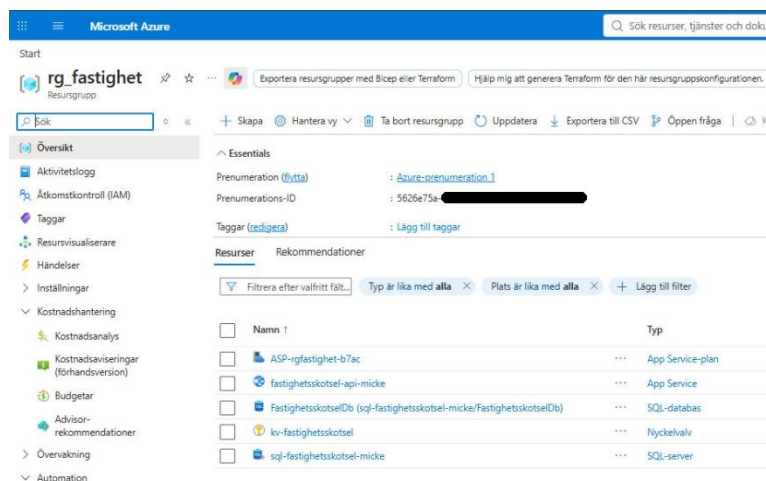


Bild 6 – Resource Group

Bilden visar de Azure-resurser som ingår i lösningen.

API:t publicerades därefter till en Azure App Service med Linux som operativsystem. App Service används för att köra ASP.NET Core-applikationen i Azure och gör API:t tillgängligt via en HTTPS-adress. Den lokala applikationen publicerades från Visual Studio genom en Publish Profile med Zip Deploy.

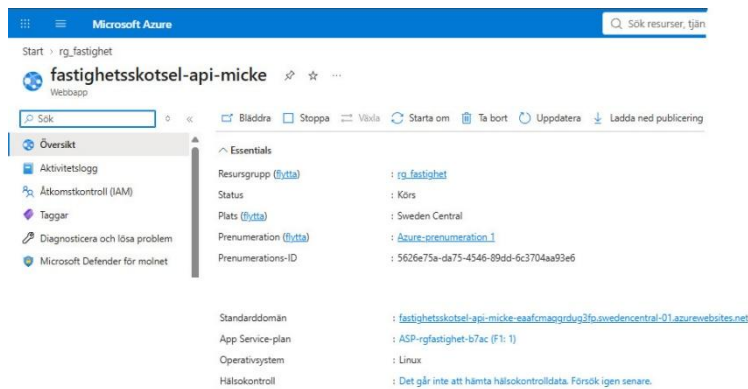


Bild 7 – Azure App Service

Bilden visar App Service fastighetssktsel-api-micke i Azure Portal.

Vid publiceringen behövde produktionsmiljön konfigureras separat från den lokala utvecklingsmiljön. Lokalt användes appsettings.Development.json för utvecklingsspecifik konfiguration, men denna fil används inte automatiskt i produktionsmiljön.

De konfigurationsvärden som API behövde för Microsoft Entra ID och andra Azure-resurser lades därför in i Azure App Service Configuration. På så sätt kunde produktionsmiljön få sina egna konfigurationsvärden utan att dessa behövde hårdkodas i applikationen eller läggas in i versionshanteringen.

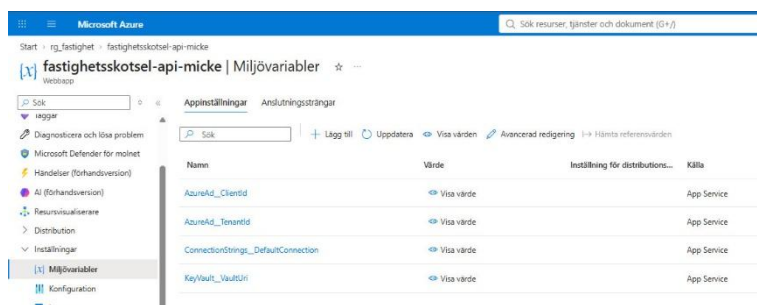


Bild 8 – App Service Configuration

Bilden visar Azure App Service → Settings → Environment variables/Configuration.

Efter att App Service och produktionskonfigurationen hade satts upp fortsatte arbetet med att konfigurera den databas som API skulle använda i Azure.

3.4 Azure SQL och databasanslutning

För att lagra felanmälningarna i produktionsmiljön skapades en Azure SQL Server och en SQL-databas med namnet FastighetssktselDb. Databasen ersätter den lokala utvecklingsdatabasen när API:t körs i Azure. Azure SQL Servern har namnet sql-fastighetssktsel-micke och används som databasserver för lösningen. Genom att använda en hanterad databastjänst i Azure behövs ingen egen installation eller drift av en SQL Server.

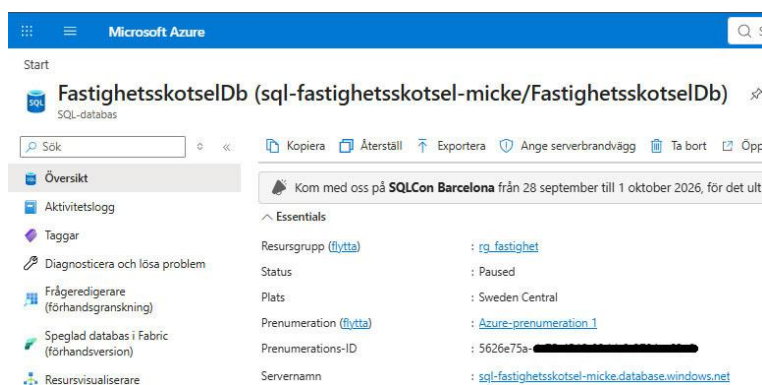


Bild 9 – Azure SQL Database

Bilden visar Azure SQL-databasen FastighetsskotselDb och tillhörande SQL Server.

För att API ska kunna kommunicera med databasen behöver nätverksåtkomsten till SQL Servern konfigureras. I denna lösning är offentlig nätverksåtkomst aktiverad. Det innebär inte att databasen är helt öppen för alla anslutningar, eftersom Azure SQL samtidigt använder en brandvägg som begränsar vilka IP-adresser som får ansluta.

Offentlig åtkomst valdes eftersom lösningen inte använder ett virtuellt Azure-nätverk med exempelvis Private Endpoint. API körs i Azure App Service och databasen exponeras därför via sin publika SQL-endpoint. Detta gör lösningen enklare att konfigurera och är tillräckligt för den aktuella uppgiften, men innebär samtidigt en större potentiell attackyta än en lösning där databasen endast är åtkomlig via ett privat nätverk.

En brandväggsregel skapades för den publika IP-adress som användes från utvecklingsmiljön. Regeln gjorde det möjligt att ansluta till databasen från den egna utvecklingsdatorn, exempelvis för att köra Entity Framework Core-migrationen mot Azure SQL och kontrollera databasen. Genom att endast tillåta den aktuella IP-adressen begränsades den externa åtkomsten istället för att öppna databasen för godtyckliga IP-adresser.

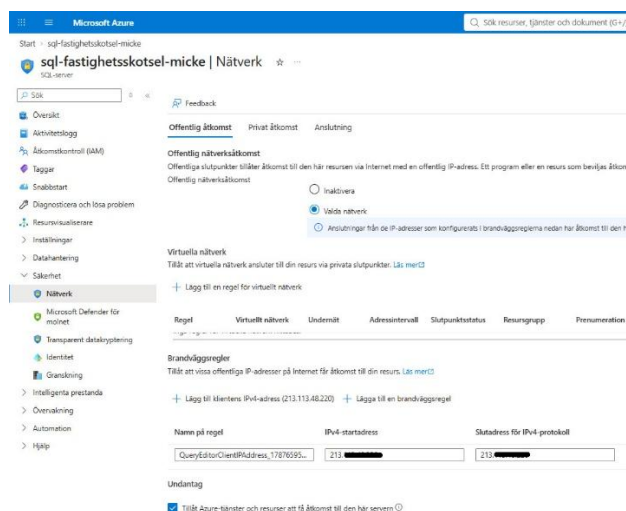


Bild 10 – SQL Server Networking

Bilden visar Azure SQL Server → Networking, där relevanta nätverks- och brandväggs-inställningar visas.

3.4.1 Databasmigration

När Azure SQL-databasen hade skapats behövde databasstrukturen från den lokala utvecklingsmiljön överföras till den nya databasen. Databasstrukturen hanteras med Entity Framework Core migrations, vilket gör att förändringar i datamodellen kan versionshanteras tillsammans med applikationen.

Den första migrationen, InitialCreate, skapades lokalt i Visual Studio utifrån den aktuella datamodellen. Därefter kördes migrationen mot Azure SQL genom Entity Framework Cores Update-Database-kommando. Vid körningen angavs en connection string som pekade mot den Azure SQL-databas som skulle användas i produktionsmiljön.

Migrationen skapade bland annat tabellen FaultReports samt tabellen __EFMigrationsHistory, som används av Entity Framework Core för att hålla reda på vilka migrationer som har applicerats på databasen. När kommandot hade körts visade resultatet att migrationen InitialCreate hade applicerats och att tabellen FaultReports hade skapats i Azure SQL. Körningen avslutades med Done, vilket bekräftade att migrationen hade genomförts utan fel.

3.4.2 Anslutning mellan API och Azure SQL

API använder en connection string med namnet DefaultConnection för sin databasanslutning. I den lokala utvecklingsmiljön pekar denna mot en LocalDB-databas, medan API i Azure ska ansluta till FastighetsskotselDb på Azure SQL Server.

Varje anslutning som används styrs av applikationens konfiguration för respektive miljö. Den produktions-specifika konfigurationen hade tidigare lagts in i Azure App Service Configuration, vilket innebär att API vid körning i Azure använder Azure SQL istället för den lokala LocalDB-databasen.

När databasen, nätverksåtkomsten och migrationen var på plats hade API därmed förutsättningar att ansluta till Azure SQL och använda tabellen FaultReports för att lagra och hämta felanmälningar.

3.5 Hemlighetshantering och konfiguration

En viktig del av lösningen var att skilja mellan applikationens kod och den konfiguration som används i olika miljöer. Lokalt används bland annat appsettings.Development.json, medan produktionsmiljön använder konfiguration som tillhandahålls av Azure.

Den produktions-specifika konfigurationen, bland annat inställningar för Microsoft Entra ID och anslutningen till Azure SQL, konfigurerades tidigare i Azure App Service Configuration. På så sätt behöver dessa värden inte hårdkodas i applikationens källkod eller lagras i Git-repositoryt.

För den hemlighet som används av SMSNotifier används istället Azure Key Vault. Hemligheten lagras i Key Vault och hämtas av API:t när SMS-tjänsten behöver använda den. Själva secret-värdet finns därmed inte i applikationens källkod eller i den versionshanterade konfigurationen.

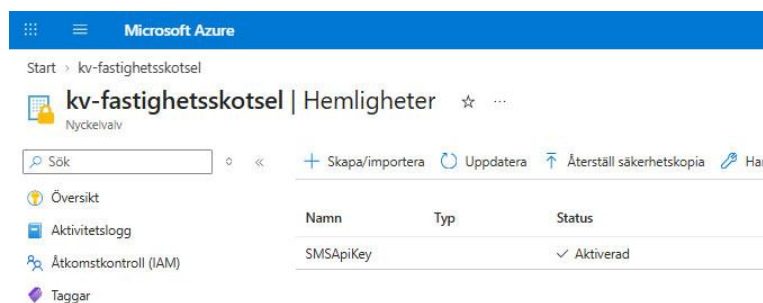


Bild 11 – Azure Key Vault

Bilden visar Azure Key Vault där secret för SMSNotifier visas (inte själva secret-värdet)

API:t använder sin Azure-identitet för att få åtkomst till Key Vault. Detta innebär att applikationen inte behöver lagra ytterligare inloggningsuppgifter för att kunna hämta hemligheten.

Lösningen använder därmed två olika mekanismer. App Service Configuration används för miljöspecifik applikationskonfiguration, medan Key Vault används för den hemlighet som behöver skyddas särskilt.

I en vidareutveckling skulle även fler känsliga konfigurationsvärden kunna flyttas till Key Vault. Detta skulle minska mängden känslig information som hanteras direkt som App Service-konfiguration och ge en mer centraliserad hantering av secrets.

3.6 Autentisering och framtagning av access token

För att kunna testa API:ets autentisering och behörighet behövdes en giltig access token utfärdad av Microsoft Entra ID. Eftersom lösningen inte innehåller någon egen frontend genomfördes inloggningen via Scalar, som även användes för att få fram en giltig access token för testning.

API:t startades lokalt från Visual Studio och Scalar öppnades i en webbläsare. Genom att välja Authorize i Scalar initierades inloggningen mot Microsoft Entra ID. Efter genomförd inloggning kunde den utfärdade access token användas vid efterföljande API-anrop.

Den praktiska testningen av API:ets endpoints genomfördes därefter huvudsakligen med Postman, där access token skickades med i anropen. På så sätt kunde både autentisering och de rollbaserade behörigheterna testas mot de skyddade endpoints som implementerats i API:t.

Flödet kan sammanfattas som:

Microsoft Entra ID → JWT access token → Postman → ASP.NET Core API

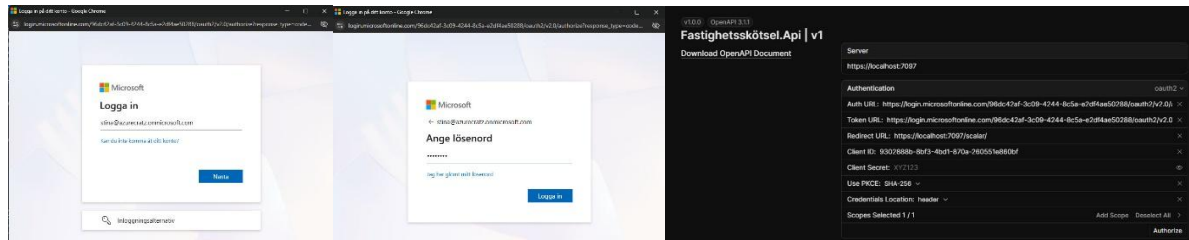


Bild 12 – Inloggning, Scalar och Microsoft Entra ID
Bilden visar en skärmdump från inloggning samt Scalar där Authorize använts.

Access token är en JWT och innehåller information som används av API vid autentisering och auktorisering. Vid kontroll av token kunde bland annat följande claims identifieras:

- aud – anger vilken API-resurs token är avsedd för.
- iss – anger vilken identitetsleverantör som har utfärdat token.
- oid – identifierar användaren i Microsoft Entra ID.
- roles – innehåller de applikationsroller som användaren har tilldelats.
- scp – anger det delegerade scope som token innehåller.

roles är särskilt relevant för API rollbaserade auktorisering eftersom värdet används för att avgöra om användaren exempelvis har rollen Resident eller PropertyManager.

Token utfärdas av Microsoft Entra ID och valideras av API. API kan därmed kontrollera både att användaren är autentiserad och att användaren har den behörighet som krävs för den aktuella endpointen.

3.6.1 Test utan autentisering

Det första testet genomfördes utan att någon access token skickades med i API-anropet. Syftet var att kontrollera att API:ets skyddade endpoints inte kunde användas av en klient som inte var autentiserad.

Ett anrop skickades till en skyddad endpoint utan giltig Authorization-header. API:t kunde därför inte identifiera någon autentiserad användare och anropet nekades.

Resultatet blev HTTP-statuskod **401 Unauthorized**.

Detta visar att autentisering krävs innan användaren kan få tillgång till den skyddade funktionen.

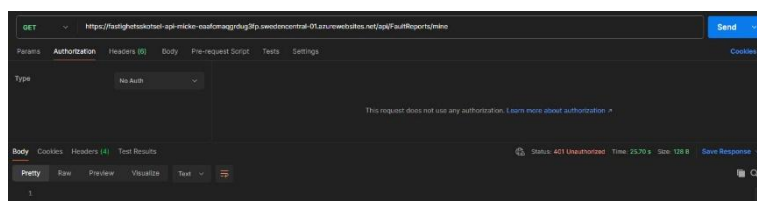


Bild 13 – 401 Unauthorized
Bilden visar en skärmdump från Postman som visar endpointen, att ingen giltig access token skickas och svaret 401 Unauthorized.

3.6.2 Test med autentisering men fel behörighet

Nästa test genomfördes med en giltig access token, men med en användare som saknade den roll som krävs för den aktuella endpointen.

Syftet var att kontrollera att API:t inte enbart kontrollerar om användaren är autentiserad, utan även kontrollerar användarens behörighet innan åtkomst beviljas.

Anropet skickades med access token för en autentiserad användare (Resident) till endpointen GET /api/FaultReports som har [Authorize(Roles = "PropertyManager")]. Token kunde valideras av API:t, men användaren hade inte den roll som krävs för den aktuella operationen. Resultatet blev HTTP-statuskod **403 Förbjuden**.

Detta visar skillnaden mellan autentisering och auktorisering. Användaren var autentiserad, vilket innebär att API:t kunde identifiera användaren, men användaren saknade tillräcklig behörighet för den begärda resursen.

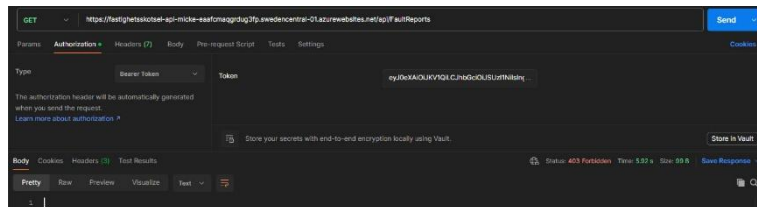


Bild 14 – 403 Förbjuden

Skärmdump från Postman som visar anropet med giltig JWT men fel roll och svaret 403 Förbjuden.

3.6.3 Test med korrekt behörighet

Det sista testet genomfördes med en användare som hade den roll som krävs för den aktuella endpointen. En giltig access token skickades med i anropet och innehöll den förväntade applikationsrollen.

För att verifiera behörigheten användes PropertyManager-användaren mot endpointen GET /api/FaultReports som har [Authorize(Roles = "PropertyManager")]

Endpointen är avsedd för att en fastighetsskötare ska kunna hämta alla felanmälningar. API:t validerade access token och kontrollerade därefter att användaren hade rätt roll.

Anropet genomfördes utan fel och API:t returnerade HTTP-statuskod **200 OK** tillsammans med användarens egna felanmälningar.

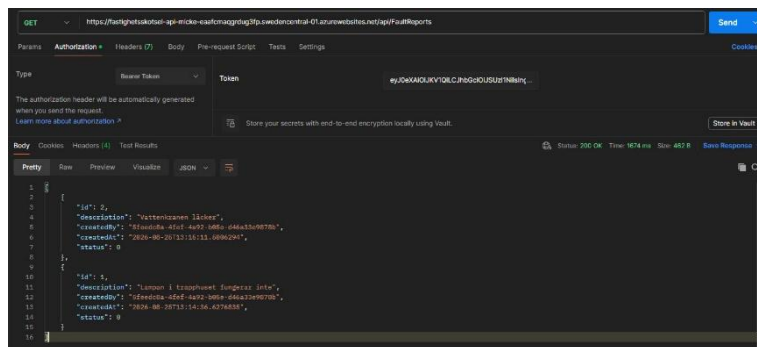


Bild 15 – 200 OK

Skärmdump från Postman som visar GET /api/FaultReports, den autentiserade användaren med rätt roll och svaret 200 OK med resultatet.

Testserien visar därmed tre olika resultat beroende på autentisering och behörighet:

Situation	Resultat
Ingen giltig access token	401 Unauthorized
Giltig token men fel roll	403 Förbjuden
Giltig token och rätt roll	200 OK

Det visar att API:t använder både autentisering och rollbaserad auktorisering för att skydda sina endpoints.

3.7 Test av rollbaserad åtkomst

Efter att autentisering och den grundläggande behörighetskontrollen hade verifierats testades API:ets rollbaserade åtkomst. Behörigheten styrs på endpointnivå med Authorize-attributet och de två applikationsrollerna Resident och PropertyManager.

Resident har behörighet att skapa felanmälningar samt att hämta sina egna felanmälningar. PropertyManager har en högre behörighetsnivå och kan hämta alla felanmälningar, hämta en specifik felanmälan samt uppdatera och ta bort felanmälningar.

Behörighetsmodellen kan sammanfattas enligt följande:

Endpoint	Resident	PropertyManager
POST /api/FaultReports	✓	✓
GET /api/FaultReports/mine	✓	–
GET /api/FaultReports	–	✓
GET /api/FaultReports/{id}	–	✓
PATCH /api/FaultReports/{id}	–	✓
DELETE /api/FaultReports/{id}	–	✓

Resident

Först testades Resident-användarens möjlighet att skapa en felanmälan. Anropet till POST /api/FaultReports genomfördes med en giltig access token innehållande rollen Resident. API:t accepterade anropet och skapade felanmälan.

Resultatet blev **201 Created**, vilket är den statuskod som endpointen returnerar när en ny felanmälan har skapats.

Därefter testades GET /api/FaultReports/mine. Anropet genomfördes med samma typ av autentiserad användare och returnerade **200 OK** tillsammans med användarens egna felanmälningar.

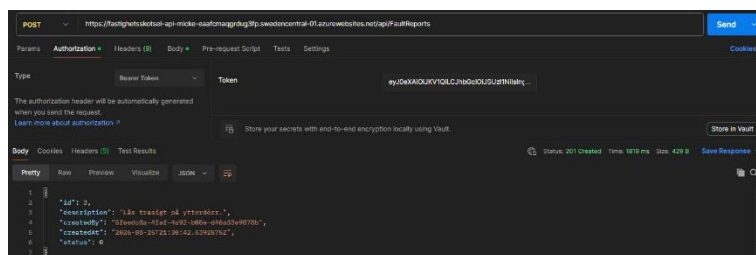


Bild 16 – Resident skapar felanmälan

Skärmdump från Postman som visar POST /api/FaultReports och svaret 201 Created tillsammans med den skapade felanmälan.

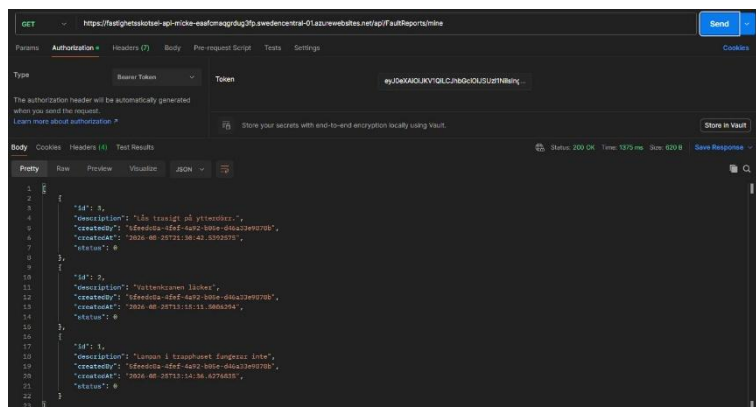


Bild 17 – Resident hämtar egna felanmälningar

Skärmdump från Postman som visar GET /api/FaultReports/mine och svaret 200 OK med resultatet.

För att verifiera att Resident inte hade tillgång till funktioner som är avsedda för fastighetsskötaren gjordes även ett försök att anropa GET /api/FaultReports. Användaren var korrekt autentiserad men saknade rollen PropertyManager.

API:t nekade därför anropet och returnerade **403 Forbidden**.

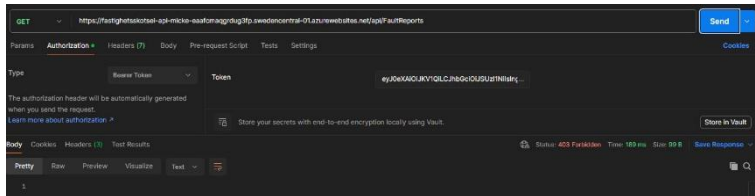


Bild 18 – Resident nekas åtkomst till alla felanmälningar

Skärmdump från Postman som visar GET /api/FaultReports och svaret 403 Forbidden med Resident-användaren.]

PropertyManager

Därefter testades PropertyManager. Användaren kunde med sin giltiga access token anropa GET /api/FaultReports och fick tillbaka samtliga felanmälningar med **200 OK**.

Detta verifierar att rollen PropertyManager ger den högre åtkomstnivå som är avsedd för fastighetsskötaren.

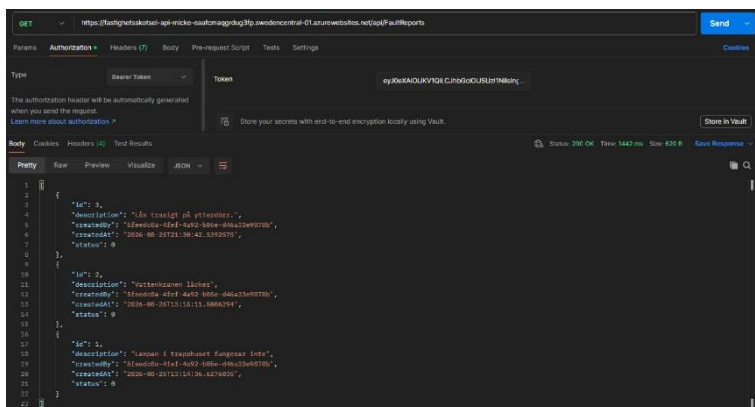


Bild 20 – PropertyManager hämtar alla felanmälningar

Skärmdump från Postman som visar GET /api/FaultReports och svaret 200 OK med resultatet.

PropertyManager har även behörighet att hämta en specifik felanmälan, uppdatera en felanmälan, skapa en felanmälan samt ta bort en felanmälan. Dessa endpoints använder samma rollbaserade auktoriseringsprincip genom att kräva rollen PropertyManager.

För att undvika att göra rapporten till ett detaljerat testprotokoll testades inte varje kombination av endpoint och roll separat. Behörighetsmodellen verifierades istället genom representativa tester som täcker båda rollerna, både tillåten och nekad åtkomst samt de olika typerna av operationer.

Testerna visar därmed att rollbaserad auktorisering fungerar enligt den avsedda modellen: en Resident har tillgång till funktioner som krävs för den boendes uppgifter, medan PropertyManager har de utökade behörigheter som krävs för administration av felanmälningar.

4. Problem som uppstod och hur de löstes

Under utvecklingen och driftsättningen av lösningen uppstod flera problem. Problemen handlade främst om skillnader mellan den lokala utvecklingsmiljön och Azure-miljön, autentisering mot Microsoft Entra ID samt anslutningen till Azure SQL.

4.1 Databasstruktur saknades i Azure SQL

Efter att API:t hade publicerats och autentiseringen fungerade testades endpointen för att hämta användarens egna felanmälningar. Anropet resulterade initialt i ett HTTP 500-fel.

Vid felsökning av loggarna framkom följande fel: *Invalid object name 'FaultReports'*

Felet visade att API:t kunde ansluta till Azure SQL-databasen, men att den tabell som API:t försökte använda inte fanns i databasen. Den lokala databasen hade tidigare skapats med Entity Framework Core migrations, men samma databasstruktur hade ännu inte skapats i Azure SQL.

Problemet löstes genom att köra Entity Framework Core-migrationen InitialCreate mot Azure SQL-databasen. Migrationen skapade bland annat tabellen FaultReports och Entity Framework Cores tabell `__EFMigrationsHistory`.

Efter att migrationen hade genomförts testades endpointen på nytt. API:t kunde då hämta data från Azure SQL och anropet returnerade **200 OK**.

Problemet visar vikten av att inte endast publicera applikationskoden vid en driftsättning. Även databasens struktur måste hanteras som en del av lösningens deploymentprocess.

4.2 Autentisering och behörighet med Microsoft Entra ID

Under konfigureringen av autentisering med Microsoft Entra ID uppstod problem när API:ets skyddade endpoints skulle testas. Ett API-anrop utan en giltig access token resulterade i **401 Unauthorized**, vilket innebar att API:t inte kunde autentisera anropet.

För att kunna testa API:t behövde därför en giltig access token erhållas från Microsoft Entra ID. Detta löstes genom att starta API:t lokalt och använda Scalar för att genomföra OAuth 2.0-autentiseringen. Via Scalar öppnades Microsoft Entra ID:s inloggningssida, där testanvändaren kunde autentiseras.

Efter en lyckad inloggning utfärdade Microsoft Entra ID en access token som kunde användas vid tester av API:t i Postman.

När en giltig token användes men användaren saknade den roll som krävdes av en endpoint returnerade API:t istället **403 Forbidden**. Detta visade att autentiseringen fungerade, men att auktoriseringen stoppade användaren på grund av otillräckliga behörigheter.

Genom att skilja mellan dessa två statuskoder blev det enklare att felsöka säkerhetskonfigurationen. Ett **401 Unauthorized** indikerade ett problem med autentisering eller avsaknad av en giltig token, medan **403 Forbidden** visade att användaren hade autentiserats men saknade den behörighet som endpointen krävde.

När rätt testanvändare, giltig access token och korrekt applikationsroll användes fungerade API-anropen och returnerade förväntade resultat.

4.3 API-dokumentation och 404-fel i Azure

Efter att API:t hade publicerats till Azure testades API:ets URL. När API:ets grundadress öppnades i webbläsaren returnerades HTTP-statuskod **404 Not Found**.

Detta berodde inte på att API:t hade misslyckats att starta. API:t var en ASP.NET Core Web API-applikation och hade ingen endpoint eller startsida konfigurerad på rotadressen /. Ett 404-svar på API:ets grundadress innebar därför inte automatiskt att publiceringen hade misslyckats.

Vidare upptäcktes att OpenAPI och Scalar endast konfigurerades när applikationen kördes i utvecklingsmiljön:

```
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}
```



```
    app.MapScalarApiReference();  
}
```

När API:t kördes i Azure användes produktionsmiljön, vilket innebar att dessa endpoints inte registrerades. Därför kunde Scalar inte nås via den publicerade API-adressen.

För att kunna använda Scalar i Azure hade OpenAPI och Scalar behövt konfigureras även för produktionsmiljön. I denna lösning användes istället den lokala utvecklingsmiljön för att genomföra OAuth 2.0-inloggningen och erhålla en access token för tester.

Detta visar vikten av att förstå skillnaden mellan en applikations olika miljöer. Konfiguration som ligger inom `IsDevelopment()` blir endast tillgänglig när applikationen körs med utvecklingsmiljön aktiverad.

I en komplett applikation skulle autentiseringen normalt initieras från en frontend eller annan klientapplikation. Användaren skulle då skickas till Microsoft Entra ID för inloggning, varefter klientapplikationen får en access token som skickas med vid anrop till API:t.

I denna lösning finns ingen separat frontend, eftersom uppgiften fokuserar på utveckling och säkerhet av själva API:t. Scalar användes därför som en testklient för att initiera OAuth 2.0-inloggningen och erhålla en access token från Microsoft Entra ID.

Den lokala Scalar-instansen ersatte därmed tillfälligt den roll som en frontend eller annan klientapplikation normalt skulle ha vid autentisering och testning av API:t. Den erhållna access token kunde sedan användas i Postman för att testa det publicerade API:ts autentisering och behörighetskontroller.

4.4 Problem med Azure-konto och autentisering mot Azure SQL

Under arbetet med Azure SQL uppstod problem med autentiseringen. Utvecklingsmiljön innehöll flera olika identiteter eftersom kursens Azure-miljö och den Azure-prenumeration som användes för den färdiga lösningen låg i olika tenants.

Vid försök att ansluta till Azure SQL med Microsoft Entra-baserad autentisering användes initialt en autentiseringsmetod som försökte använda den aktuella Windows-identiteten. Detta fungerade inte eftersom den inloggade Windows-användaren inte motsvarade en användare med rätt åtkomst i Azure SQL.

Även Azure CLI behövde vara inloggad mot rätt tenant och prenumeration. När CLI var kopplad till fel konto eller tenant gick det inte att hitta den prenumeration där Azure-resurserna fanns.

Problemet löstes genom att logga in mot rätt Microsoft Entra-tenant och använda den Azure-prenumeration där lösningens resurser var skapade. Därefter kunde Entity Framework Core ansluta till Azure SQL med Active Directory Default och migrationen kunde köras mot produktionsdatabasen.

Detta illustrerade en viktig aspekt av Microsoft Entra ID: det räcker inte att vara inloggad med ett Microsoft-konto. Identiteten måste också finnas i rätt tenant och ha de behörigheter som krävs för den aktuella Azure-resursen.

Efter att rätt konto och tenant användes fungerade anslutningen till Azure SQL och databasmigrationen kunde genomföras.

4.5 Skillnader mellan utvecklings- och produktionsmiljö

En återkommande utmaning under utvecklingen var att den lokala utvecklingsmiljön och Azure-miljön inte var identiska. Lokalt användes bland annat SQL Server LocalDB, medan den publicerade applikationen använde Azure SQL. Även konfigurationen för Microsoft Entra ID, anslutningssträngar och hemligheter skiljde sig mellan miljöerna.

Det innebar att en funktion kunde fungera lokalt men ändå kräva ytterligare konfiguration efter publicering. Ett exempel var databasen, där den lokala migrationen hade skapats och applicerats mot LocalDB men även behövde appliceras mot Azure SQL innan API:t kunde använda produktionsdatabasen.

Även API-dokumentationen påverkades av miljöskillnaden. OpenAPI och Scalar var initialt konfigurerade för utvecklingsmiljön genom `IsDevelopment()`. När API:t kördes i Azure befann det sig i produktionsmiljön och dessa delar av applikationen aktiverades därför inte.

För att hantera skillnaderna separerades miljöspecifik konfiguration från själva applikationskoden. Lokala inställningar användes under utveckling, medan produktionsinställningar konfigurerades i Azure App Service. Azure Key Vault användes dessutom för den hemlighet som behövde skyddas särskilt.

Problemen visade att en fungerande lokal utvecklingsmiljö inte automatiskt innebär att applikationen är färdig för produktion. Vid driftsättning behöver även beroenden, databaser, identiteter, behörigheter, nätverk och miljöspecifik konfiguration verifieras.

Genom att stegvis kontrollera dessa delar kunde lösningen flyttas från den lokala utvecklingsmiljön till Azure och därefter verifieras mot den riktiga produktionsmiljön.

5. Fördjupad säkerhetsanalys

5.1 Least Privilege

Principen om **Least Privilege** innebär att användare, applikationer och andra komponenter endast ska ha de behörigheter som krävs för att utföra sina uppgifter. Om en identitet skulle komprometteras begränsas därmed den skada som kan uppstå.

I lösningen används två applikationsroller: Resident och PropertyManager. Roller används för att begränsa vilka API-endpoints respektive användare får använda.

Resident har endast tillgång till de funktioner som krävs för en boende. Rollen kan skapa en felanmälan och hämta sina egna felanmälningar. PropertyManager har ytterligare behörigheter eftersom rollen behöver kunna administrera felanmälningar. Den kan därför även hämta alla felanmälningar, hämta en specifik felanmälan, uppdatera och ta bort felanmälningar, dock inte skapa nya.

Behörigheten begränsas direkt på respektive endpoint genom Authorize-attributet. En användare med rollen Resident kan exempelvis inte anropa endpointen för att hämta samtliga felanmälningar, eftersom denna kräver rollen PropertyManager.

En viktig del av principen finns även i endpointen `GET /api/FaultReports/mine`. API:t använder användarens oid från den validerade access token för att identifiera användaren. Klienten behöver därför inte skicka med ett användar-ID för att ange vilka felanmälningar som ska hämtas. Detta begränsar möjligheten för en användare att försöka hämta en annan användares felanmälningar genom att ändra ett ID i anropet.

Bedömning

Lösningen följer Least Privilege på en grundläggande nivå genom att rollerna har tydligt avgränsade behörigheter och att åtkomst kontrolleras på endpointnivå.

En möjlig förbättring vore att även tillämpa principen på Azure-resursernas behörigheter. Exempelvis bör API:ts identitet endast få de rättigheter i Azure som faktiskt krävs för att läsa den aktuella hemligheten från Key Vault, istället för bredare administratörsbehörigheter.

På motsvarande sätt bör åtkomsten till Azure SQL och övriga Azure-resurser begränsas till de operationer som respektive identitet behöver utföra. På så sätt tillämpas Least Privilege inte bara på API-användarna utan genom hela lösningens infrastruktur.

5.2 Defense in Depth

Defense in Depth innebär att säkerheten byggs upp av flera oberoende eller kompletterande skyddslager. Om ett säkerhetslager skulle kringgå ska nästa lager fortfarande kunna begränsa angriparens möjligheter.

Lösningen använder flera sådana lager.

Det första lagret är **Microsoft Entra ID**, som används för att autentisera användare. API:t accepterar därmed inte godtyckliga användaruppgifter utan kräver en giltig access token utfärdad av den konfigurerade identitetsleverantören.

Nästa lager är **valideringen av JWT-token**. API:t kontrollerar att token är giltig och att den är avsedd för rätt API. Det innebär att en token som inte uppfyller kraven inte kan användas för att komma åt de skyddade endpoints.

Därefter kommer **rollbaserad auktorisering**. Att en användare är autentiserad innebär inte automatiskt att användaren får utföra alla operationer. API:t kontrollerar vilken roll användaren har och begränsar åtkomsten genom Authorize-attributen.

Ett ytterligare lager finns i hanteringen av användarens egna felanmälningar. Endpointen GET /api/FaultReports/mine använder användarens oid från access token istället för ett användar-ID som klienten själv skickar. Därmed bygger identifieringen av användaren på den autentiserade identiteten och inte på information som klienten fritt kan ändra.

Även infrastrukturen bidrar med ytterligare säkerhetslager. API:t körs i Azure App Service och databasen ligger i Azure SQL. Kommunikation med Azure SQL sker krypterat och databasen är dessutom skyddad genom nätverksregler. För hemligheter används Azure Key Vault istället för att placera SMS-hemligheten direkt i källkoden.

Bedömning

Lösningen använder flera säkerhetsmekanismer som kompletterar varandra. Ett fel i en enskild mekanism behöver därför inte automatiskt innebära att en angripare får full tillgång till systemet.

Samtidigt finns förbättringsmöjligheter. Nätverksskyddet är exempelvis begränsat eftersom API:t har offentlig åtkomst och Azure SQL måste vara nåbar från API:t. En mer avancerad lösning skulle kunna använda privata nätverksanslutningar, exempelvis Private Endpoint och Virtual Network, för att minska exponeringen mot det publika nätverket.

Även loggning och övervakning skulle kunna förstärkas. Om ett försök att kringgå behörighetskontroller upptäcks bör detta kunna identifieras genom loggar och övervakning. På så sätt kompletteras de förebyggande säkerhetsåtgärderna med möjlighet att upptäcka och reagera på angrepp.

Sammanfattande bedömning

Lösningen följer Defense in Depth genom att kombinera identitetshantering, tokenvalidering, rollbaserad auktorisering, användarbaserad åtkomstkontroll, krypterad kommunikation, nätverksskydd och hemlighetshantering. Säkerheten är därmed inte beroende av en enda kontrollmekanism.

5.3 Secure by Design

Secure by Design innebär att säkerhet ska vara en del av systemets arkitektur och design från början, istället för att läggas till som en separat funktion i efterhand.

I lösningen är autentisering och auktorisering en integrerad del av API:ets design. Skyddade endpoints är uttryckligen markerade med Authorize och kräver rätt applikationsroll. Behörighetskontrollen sker därmed som en del av API:ets normala request-flöde.

En viktig designprincip är att användarens identitet hämtas från den validerade access token. I endpointen GET /api/FaultReports/mine används användarens oid från token för att identifiera vilka felanmälningar som ska hämtas. Klienten behöver inte skicka med ett användar-ID som API:t sedan måste lita på.

Detta är säkrare än en modell där klienten exempelvis skickar: GET /api/FaultReports/mine?userId=123 och servern använder detta värde för att avgöra vilken användares information som ska returneras. I den implementerade lösningen kommer identiteten istället från den autentiserade användaren.

Även applikationens lagerindelning bidrar till en säkrare design. Controllerlagret hanterar HTTP-anrop och behörighetskontroller medan affärslogiken ligger i service-lagret och dataåtkomsten i repository/data access-lagret. Detta gör ansvarsfördelningen tydligare och minskar risken att säkerhetsrelaterad eller affärsmässig logik sprids okontrollerat mellan olika delar av applikationen.

Även hanteringen av konfiguration är en del av designen. Produktionsspecifika värden ligger utanför källkoden och den känsliga SMS-hemligheten hanteras via Azure Key Vault. Detta minskar risken att hemligheter oavsiktligt följer med källkod och versionshantering.

Bedömning

Lösningen har säkerhet integrerad i flera delar av arkitekturen och följer därför Secure by Design på en grundläggande nivå.

En möjlig förbättring vore att utforma ännu fler säkerhetskontroller som en del av applikationens arkitektur. Exempelvis skulle åtkomst till enskilda resurser kunna styras med mer detaljerad resursbaserad auktorisering om framtida krav innebär att olika PropertyManagers endast får hantera vissa fastigheter.

Även validering av inkommande data, centraliserad felhantering och strukturerad säkerhetsloggning kan byggas in som gemensamma komponenter i arkitekturen. Det minskar risken för att en enskild endpoint utvecklas utan samma säkerhetsnivå som resten av API:t.

5.4 Secure by Default

Secure by Default innebär att systemets standardbeteende ska vara så säkert som möjligt. Säkerhet ska inte kräva att användaren eller utvecklaren aktivt måste komma ihåg att aktivera skyddet varje gång en ny funktion används.

I API:t är de funktioner som innehåller skyddsvärd information eller utför förändringar skyddade med autentisering och rollbaserad auktorisering. Endpoints använder Authorize-attribut för att ange vilka roller som krävs. En användare som saknar giltig autentisering eller rätt roll får därför inte automatiskt tillgång till funktionerna.

Även separationen mellan utvecklings- och produktionskonfiguration bidrar till principen. Produktionsspecifika värden konfigureras i Azure istället för att hårdkodas i applikationen. Den känsliga SMS-hemligheten hanteras dessutom via Azure Key Vault.

Samtidigt finns det områden där lösningen kan förbättras. API:t har offentlig åtkomst eftersom det behöver kunna ta emot anrop från en extern klient. Det innebär att den publika åtkomsten i sig inte är begränsad till ett internt nätverk. Säkerheten måste därför i större utsträckning bygga på autentisering och auktorisering.

Ett annat förbättringsområde är API-dokumentationen. OpenAPI och Scalar är i den nuvarande lösningen kopplade till utvecklingsmiljön. Detta minskar risken att test- och dokumentationsfunktioner oavsiktligt exponeras i produktion, men innebär samtidigt att utvecklaren behöver göra ett aktivt val om dessa funktioner ska göras tillgängliga även i produktionsmiljön.

Bedömning

Lösningen följer Secure by Default genom att skyddade funktioner kräver autentisering och rätt roll utan att klienten själv behöver aktivera någon extra säkerhetsfunktion.

Den publika exponeringen av API:t innebär dock att ytterligare skydd kan vara motiverade i en produktionslösning. Exempelvis skulle rate limiting kunna införas för att begränsa mängden anrop från en enskild klient och minska risken för överbelastningsattacker. Även striktare nätverksbegränsningar skulle kunna användas om verksamhetens krav tillåter det.

En viktig princip är därför att nya endpoints som standard bör betraktas som skyddade och endast göras publikt åtkomliga om det finns ett uttryckligt behov.

5.5 Identitetshantering

Identitetshantering handlar om hur användare och applikationer identifieras och autentiseras. I lösningen används Microsoft Entra ID både för användarnas autentisering och för att ge Azure-resurser en identitet som applikationen kan använda.

Användaridentitet

Användarna autentiseras genom Microsoft Entra ID. När en användare loggar in utfärdas en access token som API:t validerar innan användaren får tillgång till skyddade endpoints.

Access token innehåller bland annat information om användarens identitet och roller. API:t använder exempelvis oid för att identifiera den autentiserade användaren och roles för att avgöra vilka funktioner användaren har behörighet till.

Denna lösning innebär att API:t inte behöver hantera eller lagra användarnas lösenord. Själva autentiseringen sker hos Microsoft Entra ID och API:t ansvarar istället för att validera den mottagna token och tillämpa de behörighetsregler som gäller för applikationen.

Applikationens identitet

Även själva API:t behöver kunna identifiera sig när det kommunicerar med andra Azure-resurser. För detta har Azure App Service konfigurerats med en **System Assigned Managed Identity**.

Managed Identity innebär att Azure tillhandahåller en identitet kopplad till App Service. Applikationen kan använda denna identitet för att autentisera mot Azure-resurser utan att behöva lagra ett användarnamn, lösenord eller client secret för detta ändamål.

I lösningen har App Servicens Managed Identity tilldelats rollen **Key Vault Secrets User** på det aktuella Azure Key Vault. Detta gör det möjligt för API:t att läsa den secret som används av SMSNotifier.

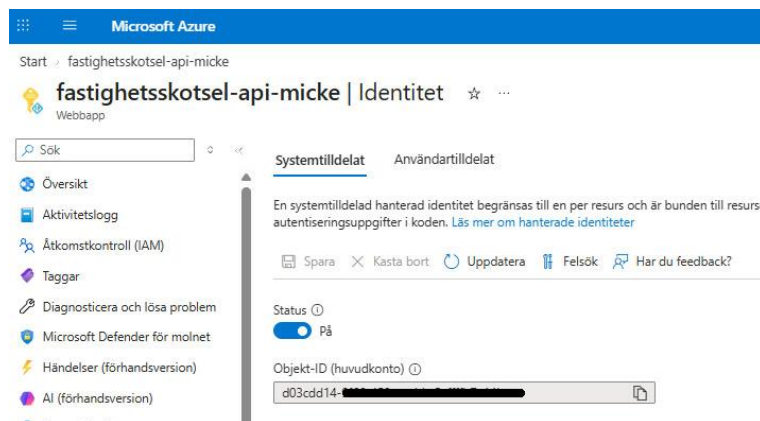


Bild 21 – System Assigned Managed Identity

Skärmdump från App Service → Identity som visar att System assigned identity är aktiverad.

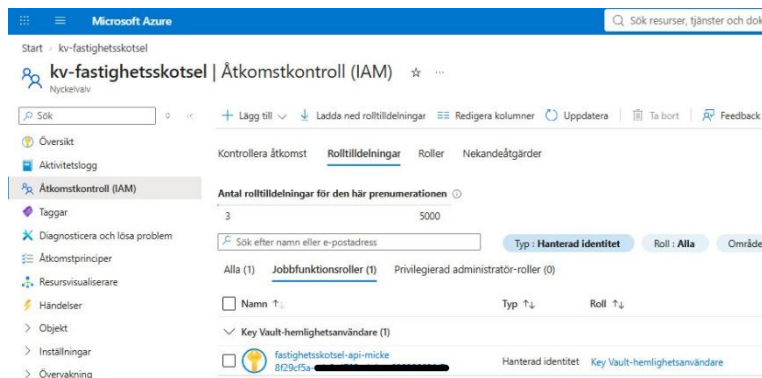


Bild 22 – Managed Identity med åtkomst till Key Vault

Skärmdump från Key Vault → Access control (IAM) som visar Key Vault Secrets User, fastighetssktsel-api-micke och att identiteten är en hanterad identitet (Managed Identity).

Det är viktigt att skilja mellan de två typerna av identitet i lösningen. Användarens identitet används för att avgöra vem som anropar API:t och vilka funktioner användaren får använda. Managed Identity används däremot för att identifiera själva API-applikationen när den behöver åtkomst till Azure-resurser.

Bedömning

Lösningen använder Microsoft Entra ID för centraliserad identitetshantering och Managed Identity för applikationens åtkomst till Azure-resurser. Detta minskar behovet av att lagra autentiseringsuppgifter i applikationen och ger möjlighet att styra åtkomsten genom Azure RBAC.

En ytterligare säkerhetsförbättring är att regelbundet granska användarnas roller och Managed Identity:ns rolltilldelningar. Behörigheter som inte längre behövs bör tas bort för att följa principen om Least Privilege.

5.6 Behörigheter

Behörigheter avgör vilka funktioner och resurser en autentiserad användare får använda. I lösningen hanteras detta genom rollbaserad auktorisering med Microsoft Entra ID och ASP.NET Cores Authorize-mekanism.

Två applikationsroller används: Resident och PropertyManager. Behörigheten anges direkt på respektive endpoint genom exempelvis:

```
[Authorize(Roles = "Resident")]
```

eller:

```
[Authorize(Roles = "PropertyManager")]
```

eller:

```
[Authorize(Roles = "Resident,PropertyManager")]
```

Det innebär att API:t inte enbart kontrollerar om användaren är inloggad. Även användarens roll måste motsvara den behörighet som krävs för endpointen.

Resident har behörighet att skapa nya felanmälningar och hämta sina egna felanmälningar. PropertyManager har utökade behörigheter och kan hämta alla felanmälningar, hämta en specifik felanmälan, skapa en felanmälan samt uppdatera och ta bort felanmälningar.

En viktig säkerhetsdetalj är att endpointen för användarens egna felanmälningar identifierar användaren genom oid från den validerade access token. Klienten behöver därför inte ange vilken användare som informationen tillhör. API:t använder den autentiserade identiteten som grund för åtkomsten.

Detta minskar risken för **Broken Object Level Authorization (BOLA)**, där en användare försöker komma åt en annan användares data genom att manipulera ett ID i ett API-anrop.

Behörighetsmodellen har verifierats genom tester där en autentiserad Resident fick tillgång till sina egna funktioner men nekades åtkomst till PropertyManagers funktioner. När en användare saknade rätt roll returnerade API:t 403 Forbidden.

Bedömning

Den implementerade rollbaserade auktoriseringen ger en tydlig och enkel behörighetsmodell som passar lösningens begränsade krav.

En möjlig förbättring är att införa mer detaljerad **policybaserad eller resursbaserad auktorisering** om systemet skulle växa. Exempelvis skulle olika PropertyManagers kunna begränsas till vissa fastigheter eller områden istället för att alla med rollen PropertyManager får tillgång till samtliga felanmälningar.

Det skulle ge en mer detaljerad behörighetsmodell än enbart roller och följer principen om Least Privilege ytterligare.

En annan förbättring vore att komplettera rollkontrollerna med mer detaljerad kontroll av vilka operationer respektive roll får utföra. Detta blir särskilt viktigt om fler roller eller funktioner läggs till i framtiden.

5.7 Nätverk

Nätverkssäkerhet handlar bland annat om att begränsa vilka system och nätverk som kan kommunicera med applikationens resurser.

I lösningen används Azure App Service för API:t och Azure SQL för databasen. API:t behöver kunna kommunicera med Azure SQL över nätverket för att kunna läsa och skriva felanmälningar.

Azure SQL konfigurerades med offentlig nätverksåtkomst. Anledningen är att API:t körs i Azure App Service och i den aktuella lösningen inte är placerat i ett privat Azure-nätverk med exempelvis Virtual Network och Private Endpoint. Offentlig åtkomst gjorde därför att App Service kunde ansluta till databasen utan att ytterligare privat nätverksinfrastruktur behövde byggas.

Den offentliga åtkomsten innebär däremot inte att vem som helst automatiskt får tillgång till databasen. Azure SQL:s brandvägg används för att begränsa vilka IP-adresser som får ansluta. Under utvecklingen skapades därför en brandvägsregel för den aktuella publika IP-adressen för att möjliggöra administrativ åtkomst till databasen från utvecklingsmiljön.

API:t autentiserar sig mot databasen med den konfigurerade identiteten och anslutningen sker krypterat. Databasen är därmed inte exponerad med obegränsad nätverksåtkomst trots att offentlig åtkomst är aktiverad.

Säkerhetsrisk

Den huvudsakliga risken med offentlig nätverksåtkomst är att databasen är exponerad mot Internet och därmed kan utsättas för exempelvis portskanning och försök till obehörig åtkomst. Brandväggen minskar denna risk, men innebär inte samma isolering som ett privat nätverk.

Det är också viktigt att brandvägsregler inte görs bredare än nödvändigt. En regel som tillåter 0.0.0.0 – 255.255.255.255 skulle exempelvis innebära att Azure SQL accepterar nätverksanslutningar från alla publika IP-adresser och skulle därför kraftigt öka attackytan.

Förbättring

En säkrare produktionsarkitektur skulle kunna använda ett **Azure Virtual Network** tillsammans med **Private Endpoint** för Azure SQL. Databasen skulle då kunna nås via en privat nätverksadress istället för att vara exponerad genom en publik endpoint.

Det skulle minska den publika attackytan och ge ett ytterligare säkerhetslager. Nackdelen är att lösningen blir mer komplex och kräver ytterligare Azure-konfiguration.

För den aktuella utbildningslösningen är offentlig åtkomst med en begränsad brandväggsregel därför en rimlig kompromiss mellan säkerhet, komplexitet och kostnad. För en produktionsmiljö med högre säkerhetskrav skulle privat nätverksåtkomst vara att föredra.

5.8 Kryptering

Kryptering används för att skydda information så att den inte kan läsas av obehöriga om kommunikationen eller lagringen skulle avlyssnas eller komma i orätta händer.

I lösningen används HTTPS för kommunikationen mellan klient och API. API:t exponeras via en HTTPS-adress och de API-anrop som genomförts i Postman använder HTTPS. Detta skyddar access tokens, felanmälningar och annan information som skickas mellan klienten och API:t under transport.

Även kommunikationen mellan API:t och Azure SQL sker krypterat. Connection stringen för produktionsmiljön är konfigurerad med `Encrypt=True` och `TrustServerCertificate=False`. Det innebär att anslutningen kräver kryptering och att servercertifikatet valideras istället för att certifikatet automatiskt accepteras.

Detta är särskilt viktigt eftersom både API och databas är separata Azure-resurser och kommunikationen sker över ett nätverk. Utan kryptering skulle information som skickas mellan resurserna kunna exponeras vid ett nätverksangrepp.

För data som lagras i Azure används dessutom den kryptering som tillhandahålls av Azure-tjänsterna. Azure SQL och Azure Key Vault använder kryptering för data som lagras på respektive tjänst. I den aktuella lösningen används tjänsternas standardiserade kryptering utan att någon egen krypteringslösning har implementerats i applikationskoden.

Bedömning

Lösningen har ett grundläggande skydd för både data under transport och data i vila. HTTPS skyddar kommunikationen mellan klient och API, medan den krypterade Azure SQL-anslutningen skyddar kommunikationen mellan API och databas.

En möjlig förbättring för en lösning med högre säkerhetskrav är att identifiera särskilt känsliga data och vid behov använda applikationsnivå-kryptering för dessa. Det kan exempelvis vara aktuellt om vissa fält i databasen skulle innehålla information som kräver ett extra skyddslager utöver den kryptering som Azure tillhandahåller.

Det är samtidigt viktigt att inte införa egen kryptering utan ett konkret behov. Azure-plattformens inbyggda kryptering och TLS ger redan ett starkt grundskydd och är enklare att underhålla än en egen krypteringslösning.

5.9 Hemlighetshantering

Lösningen använder Azure Key Vault för att hantera den känsliga SMSApiKey som används av SMSNotifier. Den praktiska konfigurationen och användningen av Key Vault beskrivs tidigare i rapporten i avsnitt 3.5, *Hemlighetsanvändning och konfiguration*.

Ur ett säkerhetsperspektiv är den viktigaste fördelen att själva secret-värdet inte behöver finnas i källkoden eller versionshanteringen. API:t använder istället sin System Assigned Managed Identity för att få åtkomst till Key Vault. Managed Identity har tilldelats rollen Key Vault Secrets User, vilket ger API:t den åtkomst som krävs för att läsa secrets utan att ge det administratörsbehörighet till hela Key Vault.

Detta följer både principerna **Least Privilege** och **Secure by Design**. Applikationen behöver inte lagra ytterligare credentials för att autentisera mot Key Vault, och åtkomsten kan styras centralt genom Azure RBAC.

Bedömning

Lösningen har en god grund för säker hemlighetshantering. En möjlig förbättring i en större produktionsmiljö är att införa regelbunden rotation av secrets och övervakning av åtkomsten till Key Vault.

Det bör även regelbundet kontrolleras att inga värden som faktiskt är secrets har placerats i källkod, konfigurationsfiler eller andra platser där de kan exponeras. På samma sätt bör Key Vaults rolltilldelningar granskas så att endast de identiteter som behöver åtkomst har behörighet.

För den aktuella lösningen bedöms Key Vault tillsammans med Managed Identity vara en lämplig lösning för att skydda den känsliga API-nnyckeln.

5.10 Loggning

Loggning är viktig för att kunna felsöka systemet och framför allt för att kunna upptäcka och utreda säkerhetsrelaterade händelser. Exempel på sådana händelser är misslyckade autentiseringsförsök, nekad åtkomst, ändringar av data och försök att komma åt resurser som användaren saknar behörighet till.

I den aktuella lösningen används den loggning som tillhandahålls av ASP.NET Core och Azure App Service. Under utvecklingen användes exempelvis applikationens loggar för att identifiera problem med databasanslutningen och migrationen.

Någon särskild säkerhetsloggning har däremot inte implementerats i API:t. Det innebär exempelvis att lösningen inte har en egen audit-logg som registrerar vilken användare som ändrat eller tagit bort en viss felanmälan.

Detta är en begränsning ur säkerhetsperspektiv. Om en användare med rollen PropertyManager exempelvis skulle ändra eller ta bort felaktiga poster finns det i den nuvarande lösningen ingen egen audit-logg som enkelt kan visa vem som utförde operationen och när den genomfördes.

Bedömning

Grundläggande applikationsloggning finns, men säkerheten skulle förbättras av en mer strukturerad audit-loggning.

En möjlig lösning är att registrera säkerhetsrelevanta händelser, exempelvis:

- lyckade och misslyckade autentiseringsförsök
- nekade auktoriseringsförsök
- skapande av felanmälningar
- uppdatering av felanmälningar
- borttagning av felanmälningar
- vilken användare som utförde operationen
- tidpunkt för operationen

För att identifiera användaren kan API:t använda den autentiserade användarens oid från access token. Loggarna bör däremot inte innehålla själva access token eller andra hemligheter.

I Azure skulle loggningen kunna kompletteras med **Application Insights** och centraliserad logghantering. Det skulle göra det möjligt att söka efter avvikande beteenden och skapa larm vid exempelvis upprepade nekade åtkomstförsök.

Loggning bör därför ses som ett kompletterande säkerhetslager. Den förhindrar inte i sig ett angrepp, men gör det möjligt att upptäcka, analysera och följa upp säkerhetshändelser.

5.11 Övervakning

Övervakning kompletterar loggningen genom att kontinuerligt följa applikationens och infrastrukturens tillstånd. Syftet är att kunna upptäcka avvikelser, prestandaproblem och säkerhetsrelaterade händelser och vid behov vidta åtgärder.

I den aktuella lösningen finns grundläggande möjligheter till övervakning genom Azure och App Service. Applikationens status och grundläggande information om driften kan följas i Azure-portalen.

Någon mer avancerad övervakning eller automatiserad incidenthantering har däremot inte implementerats i projektet. Det finns exempelvis inga särskilda larm som automatiskt reagerar på ett stort antal misslyckade autentiseringsförsök, upprepade 403 Forbidden eller ett ovanligt stort antal API-anrop.

Bedömning

Övervakningen skulle kunna förbättras genom att använda **Azure Application Insights** tillsammans med Azure Monitor. Application Insights kan samla in information om exempelvis requester, svarstider, fel och undantag.

Med sådan övervakning skulle det även vara möjligt att identifiera säkerhetsrelaterade mönster. Ett exempel är en kraftig ökning av 401 Unauthorized eller 403 Forbidden, vilket skulle kunna indikera automatiserade försök att komma åt API:t eller försöka kringgå behörighetskontroller.

Larm skulle kunna konfigureras för händelser som överskrider definierade tröskelvärden. Exempelvis skulle ett stort antal misslyckade anrop från samma källa kunna generera en varning som behöver undersökas.

Även Azure SQL och Key Vault bör omfattas av övervakningen i en mer avancerad produktionslösning. På så sätt kan ovanlig aktivitet mot både databasen och hemlighetshanteringen upptäckas.

Övervakning bör därför ses som ett komplement till de förebyggande säkerhetsåtgärderna i lösningen. Autentisering, auktorisering och nätverksskydd försöker förhindra obehörig åtkomst, medan övervakning gör det möjligt att upptäcka när något avviker från det normala.

Förbättring

För den aktuella lösningen skulle ett naturligt nästa steg vara att aktivera Application Insights och konfigurera relevanta Azure Monitor-larm. Detta skulle ge bättre insyn i API:ets drift och samtidigt skapa bättre förutsättningar för att upptäcka säkerhetsrelaterade incidenter.

6. Identifierade säkerhetsrisker

Utifrån säkerhetsanalysen har ett antal risker identifierats i lösningen. Riskerna är kopplade både till den nuvarande implementationen och till hur lösningen skulle kunna angripas eller missbrukas i en verklig produktionsmiljö.

För varje risk beskrivs konsekvensen av att risken utnyttjas samt möjliga åtgärder för att minska risken.

6.1 Offentlig åtkomst till Azure SQL

Risk:

Azure SQL är konfigurerad med offentlig nätverksåtkomst. Även om brandväggen begränsar vilka IP-adresser som får ansluta innebär detta att databasen har en publik nätverksyta.

Konsekvens:

En publik databasendpoint kan utsättas för exempelvis portskanning och försök till obehörig åtkomst. Om en angripare dessutom skulle få tillgång till giltiga autentiseringsuppgifter eller på annat sätt kringgå nätverksskyddet kan databasen utsättas för ytterligare angrepp.

Riskreducering:

Databasen kan placeras bakom en privat nätverksanslutning med Azure Virtual Network och Private Endpoint. Då behöver Azure SQL inte exponeras publikt. Om offentlig åtkomst används bör brandvägsreglerna hållas så restriktiva som möjligt.

6.2 Komprometterad användare med rollen Resident

Risk:

Om en angripare lyckas ta över en användares Microsoft Entra ID-konto kan angriparen använda den användarens access token för att anropa API:t.

Konsekvens:

En angripare kan då utföra de operationer som rollen Resident har behörighet till, exempelvis skapa felanmälningar och hämta den komprometterade användarens egna felanmälningar.

Riskreducering:

Microsoft Entra ID kan kompletteras med exempelvis multifaktorautentisering och Conditional Access. Tokenlivslängder och sessionshantering kan också konfigureras utifrån verksamhetens säkerhetskrav.

Den begränsade behörigheten för Resident minskar samtidigt konsekvensen jämfört med om samma användare hade haft administrativa rättigheter.

6.3 Komprometterad PropertyManager

Risk:

En användare med rollen PropertyManager har betydligt större behörigheter än en Resident. Om ett sådant konto komprometteras får angriparen tillgång till funktioner för att läsa, ändra och ta bort felanmälningar.

Konsekvens:

Angriparen kan exempelvis läsa information om samtliga felanmälningar, ändra information eller ta bort poster. Detta kan påverka både informationens konfidentialitet och integritet.

Riskreducering:

Administrativa roller bör tilldelas så få användare som möjligt. Multifaktorautentisering och Conditional Access bör användas för privilegierade användare. Behörigheter bör regelbundet granskas och tas bort när de inte längre behövs.

En ytterligare åtgärd är att införa audit-loggning så att ändringar och borttagningar kan kopplas till den användare som utförde operationen.

6.4 Otillräcklig loggning och övervakning

Risk:

Den nuvarande lösningen har inte någon egen detaljerad audit-loggning för säkerhetskritiska operationer.

Konsekvens:

Om exempelvis en felanmälan ändras eller tas bort på ett obehörigt eller misstänkt sätt kan det vara svårt att i efterhand avgöra exakt vad som hände, vem som utförde operationen och när det skedde.

Det kan även göra det svårare att upptäcka pågående angrepp.

Riskreducering:

Säkerhetsrelevanta händelser bör loggas strukturerat och kompletteras med Azure Monitor och Application Insights. Larm kan exempelvis konfigureras för ovanligt många misslyckade autentiserings- eller auktoriseringsförsök.

Loggarna bör utformas så att känslig information, exempelvis access tokens och secrets, inte skrivs till loggen.

6.5 Exponering av hemligheter

Risk:

En API-nyckel eller annan secret kan orsaka en säkerhetsincident om den av misstag placeras i källkod, versionshantering eller vanlig konfiguration och därefter exponeras.

Konsekvens:

En angripare som får tillgång till en giltig secret kan använda den för att få tillgång till den bakomliggande tjänsten. Beroende på vilken tjänst secret tillhör kan detta exempelvis innebära kostnader, informationsläckage eller missbruk av tjänsten.

Riskreducering:

I lösningen används Azure Key Vault för SMSApiKey, vilket minskar denna risk. App Service använder sin Managed Identity för att läsa secret-värdet och behöver därför inte lagra separata credentials för Key Vault.

Som ytterligare skydd bör secrets kunna roteras och gamla värden återkallas. Åtkomst till Key Vault bör också begränsas enligt Least Privilege och övervakas.

Sammanfattande bedömning

De identifierade riskerna visar att lösningen har flera säkerhetslager men att det fortfarande finns områden där säkerheten kan förstärkas. Framför allt är offentlig nätverksåtkomst till Azure SQL samt begränsad loggning och övervakning områden där ytterligare säkerhetsåtgärder skulle kunna ge ett starkare skydd.

Samtidigt minskar flera av de implementerade säkerhetsåtgärderna konsekvenserna av ett eventuellt angrepp. Rollbaserad auktorisering begränsar vad en komprometterad användare kan göra, medan Key Vault och Managed Identity minskar risken för exponerade credentials.

7. Föreslagen säkerhetsförbättring

Utifrån den genomförda säkerhetsanalysen har Azure Private Endpoint identifierats som en säkerhetsåtgärd som skulle kunna förbättra lösningen. Åtgärden är inte implementerad i den aktuella versionen av lösningen, utan analyseras som en möjlig vidareutveckling.

7.1 Azure Private Endpoint

Azure Private Endpoint kan användas för att ge Azure SQL en privat nätverksanslutning genom ett Azure Virtual Network. Kommunikation mellan API:t och databasen kan då ske över en privat IP-adress istället för via en publik endpoint.

Den främsta säkerhetsfördelen är att databasen kan isoleras från det publika Internet. Detta minskar den publika attackytan och gör det svårare för en extern angripare att ens försöka kommunicera direkt med databasen.

Åtgärden stärker framför allt principen **Defense in Depth** genom att lägga till ytterligare ett nätverksbaserat säkerhetslager. Den bidrar även till **Least Privilege**, eftersom nätverksåtkomsten kan begränsas till de resurser som faktiskt behöver kommunicera med databasen.

Private Endpoint innebär dock en högre komplexitet. Ett Virtual Network, nätverkskonfiguration och DNS-hantering behöver konfigureras korrekt. Det innebär också mer administration jämfört med en lösning där databasen är publikt åtkomlig och skyddas med brandväggsregler.

7.2 Alternativ säkerhetsåtgärd

Ett alternativ är att fortsätta använda offentlig nätverksåtkomst men begränsa åtkomsten med mycket restriktiva brandväggsregler.

Denna lösning är enklare att konfigurera och administrera. Åtkomsten kan begränsas till specifika IP-adresser eller andra tillåtna nätverkskällor.

Säkerhetsnivån blir däremot lägre än med Private Endpoint eftersom Azure SQL fortfarande har en publik endpoint. Brandväggen begränsar vilka anslutningar som accepteras, men den publika exponeringen kvarstår.

7.3 Motivering

Private Endpoint väljs som rekommenderad säkerhetsförbättring eftersom den angriper den identifierade risken på ett mer grundläggande sätt. Istället för att enbart begränsa vilka som får ansluta till en publik endpoint minskar Private Endpoint själva behovet av publik nätverksåtkomst.

För en mindre utbildningslösning kan den ökade komplexiteten vara svår att motivera. För en produktionslösning där databasen innehåller viktig eller känslig information är däremot den minskade publika attackytan en betydande säkerhetsfördel.

Private Endpoint är därför den åtgärd som rekommenderas om lösningen skulle vidareutvecklas för en verklig produktionsmiljö med högre säkerhetskrav.

8. Slutsats

I uppgiften har ett ASP.NET Core Web API utvecklats och konfigurerats för att använda Microsoft Entra ID för autentisering och rollbaserad auktorisering. Lösningen har sedan konfigurerats och publicerats i Azure tillsammans med Azure SQL, Azure Key Vault och Azure App Service.

Säkerhetslösningen bygger på flera lager. Microsoft Entra ID används för att autentisera användare och rollerna Resident och PropertyManager begränsar vilka funktioner respektive användare får använda. API:t använder även användarens oid för att identifiera användaren vid åtkomst till egna felanmälningar.

För applikationens åtkomst till Azure-resurser används System Assigned Managed Identity. Genom att kombinera Managed Identity med Azure Key Vault kan den känsliga SMSApiKey hanteras utan att behöva placeras direkt i applikationens källkod.

Säkerhetsanalysen visar samtidigt att en lösning inte blir helt säker enbart genom att använda autentisering och roller. Nätverksskydd, kryptering, hemlighetshantering, loggning och övervakning är också viktiga delar av en säker helhet.

Den största identifierade möjligheten till vidareutveckling är nätverkssäkerheten kring Azure SQL. En framtida förbättring skulle vara att använda Azure Private Endpoint för att minska den publika exponeringen av databasen.

Sammantaget har uppgiften visat hur flera Azure-tjänster och säkerhetsprinciper kan kombineras för att skapa ett API med ett grundläggande säkerhetsskydd. Arbetet har också tydliggjort att säkerhet behöver ses som en kontinuerlig process där nya risker identifieras och lösningen förbättras i takt med att kraven förändras.

9. Användning av AI som stöd

Under arbetet med uppgiften har AI använts som ett stöd i utvecklingsprocessen och i dokumentationen. AI har bland annat använts för att diskutera arkitektur och säkerhetslösningar, förklara tekniska koncept inom .NET och Azure samt som stöd vid felsökning när problem har uppstått under utvecklingen.

AI har även använts som stöd vid strukturering och formulering av rapporten. Förslag och förklaringar har granskats och anpassats utifrån den faktiska lösningen och de steg som genomförts i projektet.

Den praktiska utvecklingen och configurationen av lösningen har genomförts och verifierats av mig. AI har fungerat som ett stöd i arbetet, men de tekniska beslut som beskrivs i rapporten bygger på den lösning som faktiskt har utvecklats och konfigurerats.

AI har därmed använts som ett hjälpmedel under arbetets gång, snarare än som ersättning för den egna utvecklingen och förståelsen av lösningen.

Bilaga A – Verktyg och metoder inom Identity and Access Management

A.1 Identity and Access Management (IAM)

Identity and Access Management (IAM) är ett samlingsbegrepp för de processer, metoder och tekniska lösningar som används för att hantera digitala identiteter och styra åtkomsten till system, applikationer och resurser.

En central del av IAM är att säkerställa att rätt person eller system identifieras och att den därefter endast får tillgång till de resurser och funktioner som den har behörighet till. IAM omfattar därför både hantering av identiteter och kontroll av åtkomst.

IAM har en viktig roll i säker molnutveckling eftersom molnapplikationer ofta består av flera tjänster och resurser som behöver kommunicera med varandra. Genom en tydlig identitets- och behörighetsmodell kan åtkomsten till dessa resurser begränsas och risken för obehörig åtkomst minskas.

I Azure kan IAM exempelvis hanteras genom Microsoft Entra ID och Azure RBAC. Microsoft Entra ID kan användas för att hantera användaridentiteter, autentisering och applikationsroller, medan Azure RBAC används för att styra vilka åtgärder en identitet får utföra på Azure-resurser.

I den aktuella lösningen används Microsoft Entra ID för att autentisera användare och tillhandahålla rollerna Resident och PropertyManager. Roller och behörigheter används därefter av API:t för att avgöra vilka operationer en autentiserad användare får utföra.

A.2 Authentication, Authorization, Identity och Role

Begreppen Authentication, Authorization, Identity och Role är centrala inom IAM, men beskriver olika delar av processen för att hantera identiteter och åtkomst.

Begrepp	Förklaring	Exempel
Authentication	Processen att verifiera att en användare eller annan identitet verkligen är den den utger sig för att vara.	En användare loggar in med Microsoft Entra ID och identiteten verifieras.
Authorization	Processen att avgöra vad en autentiserad användare eller identitet får göra och vilka resurser den får använda.	API:t kontrollerar om användaren har rollen Resident eller PropertyManager innan en endpoint får användas.
Identity	Den digitala identitet som representerar en användare, applikation eller annan entitet i ett system.	En användare i Microsoft Entra ID representeras av en identitet med bland annat ett unikt objekt-ID.
Role	En uppsättning behörigheter eller åtkomstregler som kan tilldelas en identitet.	Rollen PropertyManager ger åtkomst till att hämta, skapa, uppdatera och radera felanmälningar.

Authentication och Authorization

Skillnaden mellan Authentication och Authorization kan förenklas till två frågor:

Authentication: *Vem är du?*

Authorization: *Vad får du göra?*

I den aktuella lösningen sker Authentication genom Microsoft Entra ID. När användaren har autentiserats utfärdas en JWT-baserad access token. Token innehåller information om den autentiserade användaren och dennes tilldelade roller.

ASP.NET Core använder därefter informationen i token för Authorization. Exempelvis kan en endpoint skyddas med `[Authorize(Roles = "PropertyManager")]`, vilket innebär att endast en autentiserad användare med rollen PropertyManager får använda endpointen.

Identity och Role

Identity och Role är också två olika begrepp. Identity beskriver **vem** som försöker få åtkomst, medan Role beskriver **vilken behörighet** identiteten har.

En och samma identitet kan exempelvis tilldelas olika roller beroende på vilka arbetsuppgifter personen ska kunna utföra. I den aktuella lösningen används Microsoft Entra ID för identiteterna och App Roles för att representera Resident och PropertyManager.

Det innebär att en användares identitet först verifieras genom Authentication. Därefter används användarens tilldelade Role för Authorization och för att avgöra vilka operationer användaren får utföra.

A.3 Least Privilege

Least Privilege innebär att en användare, applikation eller annan identitet endast ska tilldelas de behörigheter som krävs för att utföra sina arbetsuppgifter, och inte mer. Behörigheterna bör även begränsas i omfattning så att en identitet endast får åtkomst till de resurser och funktioner som faktiskt behövs.

Principen är viktig eftersom konsekvenserna av en komprometterad identitet kan begränsas. Om en angripare får tillgång till en identitet med omfattande behörigheter kan angriparen potentiellt komma åt eller påverka många resurser. Med Least Privilege minskar den möjliga skadeomfattningen.

I Azure kan Least Privilege exempelvis tillämpas med Azure Role-Based Access Control (RBAC). Istället för att ge en användare rollen Owner på en hel prenumeration kan användaren tilldelas en mer begränsad roll, exempelvis Reader, och endast på den resurs eller resursgrupp där åtkomsten behövs.

Principen kan även tillämpas på applikationers identiteter. I den aktuella lösningen används en System Assigned Managed Identity för App Service. Den hanterade identiteten behöver endast de behörigheter som krävs för att applikationen ska kunna använda sina Azure-resurser, exempelvis åtkomst till den aktuella hemligheten i Azure Key Vault. Den behöver därmed inte ha administratörsbehörighet över hela Azure-miljön.

Least Privilege innebär därför inte att behörigheter ska tas bort så långt som möjligt, utan att varje identitet ska ha **precis den åtkomst som krävs för sitt uppdrag**.

A.4 Säkerhetsåtgärder vid utveckling av molnapplikationer

Vid utveckling av molnbaserade applikationer behöver säkerhet beaktas genom hela utvecklingsprocessen.

Några centrala säkerhetsåtgärder är följande.

1. Stark autentisering och tydlig behörighetskontroll

Användare bör autentiseras på ett säkert sätt och systemet bör kontrollera vilka resurser och funktioner varje användare har rätt att använda. Autentisering och auktorisering bör inte betraktas som samma sak.

I den aktuella lösningen används Microsoft Entra ID för autentisering och App Roles tillsammans med ASP.NET Cores [Authorize] för att begränsa åtkomsten till API:ets endpoints.

2. Least Privilege

Användare, applikationer och tjänster bör endast få de behörigheter som krävs för deras uppgift. Detta minskar konsekvenserna om en identitet skulle komprometteras.

I Azure kan detta exempelvis genomföras med Azure RBAC och begränsade behörigheter för Managed Identities.

3. Säker hantering av secrets

Lösenord, API-nycklar, connection strings och andra känsliga uppgifter bör inte lagras direkt i källkoden. I molnmiljöer bör de istället hanteras med lämpliga tjänster för hemlighetshantering.

I den aktuella lösningen används Azure Key Vault för lagring av SMSApiKey, och App Service använder Managed Identity för att få åtkomst till hemligheten.

4. Kryptering

Känslig information bör skyddas både när den lagras och när den överförs. Encryption at Rest skyddar data som är lagrad, medan Encryption in Transit skyddar data under kommunikation mellan olika system.

Exempelvis bör API-kommunikation ske över HTTPS och känsliga data i databasen bör skyddas genom de säkerhetsmekanismer som den använda molntjänsten tillhandahåller.

5. Nätverkssäkerhet

Åtkomsten till molnresurser bör begränsas så att endast nödvändig kommunikation tillåts. Brandväggar, nätverksregler och privata nätverksanslutningar kan användas för att minska exponeringen.

Ett exempel på en vidare säkerhetsåtgärd är att använda Private Endpoint för Azure SQL för att minska behovet av publik nätverksåtkomst.

6. Loggning och övervakning

Applikationer bör logga relevanta händelser och övervakas för att kunna upptäcka fel, misstänkta aktiviteter och säkerhetsincidenter. Loggar kan även användas vid felsökning och efteranalys av incidenter.

För verksamhetskritiska händelser kan audit-loggning användas för att skapa spårbarhet kring exempelvis vem som skapade, ändrade eller raderade information.

7. Säkerhetsuppdateringar och beroendehantering

Applikationen och dess beroenden bör hållas uppdaterade. Föråldrade ramverk, bibliotek och paket kan innehålla kända sårbarheter som kan utnyttjas av angripare.

Genom regelbundna uppdateringar och kontroll av beroenden minskar risken att applikationen innehåller kända säkerhetsproblem.

A.5 Hantering av känslig information

Känslig information som lösenord, API-nycklar, klienthemligheter och databaslösenord bör inte lagras direkt i källkoden. Om sådan information finns i källkoden kan den oavsiktligt följa med till versionshantering, delas med andra utvecklare eller exponeras om källkoden blir åtkomlig för en obehörig person.

I en molnapplikation bör känslig information istället lagras i en tjänst som är avsedd för säker hemlighetshantering, exempelvis Azure Key Vault. Applikationen kan därefter få tillgång till hemligheten genom en identitet och tilldelade behörigheter, istället för att själva hemligheten behöver finnas i applikationens källkod.

I den aktuella lösningen används Azure Key Vault för att lagra API-nyckeln som används av SMSNotifier. App Service använder en Managed Identity för att autentisera sig mot Key Vault och får endast den behörighet som krävs för att läsa den aktuella hemligheten.

Även anslutningssträngar bör hanteras med försiktighet. Konfigurationsvärden som inte är hemliga kan exempelvis ligga i appsettings.json, medan miljöspecifika värden kan konfigureras genom App Service Configuration. Om en anslutningssträng innehåller användarnamn, lösenord eller andra autentiseringsuppgifter ska dessa inte exponeras i källkoden utan hanteras som secrets.

Identifierare som exempelvis Tenant ID och Client ID är däremot inte i sig hemligheter. De används för att identifiera en Microsoft Entra-tenant respektive en appregistrering. Det är framför allt autentiseringsuppgifter såsom Client Secret och API-nycklar som måste skyddas.

En säker princip är därför att skilja på **konfiguration som kan exponeras** och **hemligheter som måste skyddas**, samt att använda den minst privilegierade åtkomst som krävs för att applikationen ska kunna läsa sina secrets.

A.6 Defense in Depth

Defense in Depth innebär att en molnlösning skyddas genom flera olika säkerhetslager istället för att förlita sig på en enda säkerhetsåtgärd. Om ett skyddslager kringgås eller på annat sätt misslyckas ska ytterligare lager fortfarande kunna förhindra eller begränsa ett angrepp.

Principen är viktig eftersom säkerhetsåtgärder aldrig bör betraktas som felfria. Ett lösenord kan exempelvis läcka, en sårbarhet kan finnas i en applikation eller en felaktig konfiguration kan göra en resurs mer åtkomlig än avsett. Genom att kombinera flera säkerhetsmekanismer minskar risken att ett enskilt fel leder till ett fullständigt intrång.

En molnapplikation kan exempelvis använda flera lager enligt följande:

- **Identitet:** Microsoft Entra ID används för att autentisera användare.
- **Behörighet:** Roller och RBAC begränsar vad användare och tjänster får göra.
- **Applikationssäkerhet:** API:ets endpoints skyddas genom autentisering och rollbaserad auktorisering.
- **Nätverk:** Brandväggsregler och begränsad nätverksåtkomst kan minska exponeringen av resurser.
- **Hemlighetshantering:** Azure Key Vault skyddar känsliga uppgifter från att lagras direkt i källkoden.
- **Managed Identity:** Applikationer kan autentisera mot Azure-resurser utan att hantera egna credentials.
- **Kryptering:** HTTPS skyddar information under överföring och molntjänsternas krypteringsmekanismer skyddar lagrad information.

I den aktuella lösningen används flera av dessa lager tillsammans. En angripare behöver exempelvis inte bara nå API:t utan måste även kunna passera autentiseringen och ha en roll som ger tillgång till den efterfrågade

operationen. API:ets åtkomst till Key Vault skyddas dessutom separat genom Managed Identity och tilldelade behörigheter.

Defense in Depth innebär därmed att säkerheten inte är beroende av en enda kontroll. Om en säkerhetsåtgärd misslyckas kan andra lager fortfarande begränsa angriparens möjligheter och minska konsekvenserna av ett intrång.

Bilaga B – Säkerhetsproblem och åtgärder

B.1 Känslig information i källkod

Känslig information bör inte lagras direkt i källkoden eftersom källkod ofta hanteras genom versionshantering och kan delas mellan flera personer eller lagras i externa kodplattformar. Om exempelvis en API-nyckel, ett lösenord eller en klienthemlighet skrivs in direkt i källkoden finns risk att informationen exponeras för personer som inte ska ha tillgång till den.

Problemet kan dessutom kvarstå även om informationen senare tas bort från den aktuella versionen av koden, eftersom tidigare versioner kan finnas kvar i versionshistoriken.

I en molnlösning bör secrets därför hanteras separat från applikationens källkod. Azure Key Vault kan exempelvis användas för att lagra API-nycklar och andra hemligheter. Applikationen kan sedan autentisera sig mot Key Vault med en Managed Identity och endast få den behörighet som krävs för att läsa sina secrets.

Detta innebär att källkoden kan delas och versionshanteras utan att de faktiska hemligheterna behöver följa med.

B.2 Vanliga säkerhetsproblem i molnapplikationer

B.2.1 Felaktig hantering av secrets

Varför problemet uppstår

Ett vanligt problem är att utvecklare placerar API-nycklar, lösenord eller connection strings direkt i källkoden eller i konfigurationsfiler som sedan versionshanteras. Det kan exempelvis ske för att det är enkelt under utvecklingen eller för att informationen behövs för att applikationen ska kunna ansluta till andra tjänster.

Konsekvenser

Om informationen exponeras kan en angripare använda de komprometterade uppgifterna för att få åtkomst till tjänster eller data. Konsekvensen beror på vilka behörigheter den komprometterade identiteten eller nyckeln har och kan exempelvis innebära obehörig åtkomst till databaser eller externa tjänster.

Förebyggande

Secrets bör lagras i en dedikerad secret manager, exempelvis Azure Key Vault. Applikationen bör använda Managed Identity eller annan lämplig identitetsmekanism för att få åtkomst till secrets. Behörigheterna bör samtidigt begränsas enligt Least Privilege.

B.2.2 Bristande åtkomstkontroll

Varför problemet uppstår

Bristande åtkomstkontroll kan uppstå om ett API inte kontrollerar användarens identitet och behörighet tillräckligt noggrant. Ett exempel är att en endpoint endast kontrollerar att användaren är inloggad men inte vilken roll eller vilka resurser användaren har rätt att komma åt.

Konsekvenser

En angripare kan då få tillgång till funktioner eller information som egentligen är avsedd för andra användare eller roller. Det kan exempelvis innebära att en vanlig användare kan utföra administrativa operationer eller läsa andra användares information.

Förebyggande

API:ets endpoints bör skyddas med autentisering och auktorisering. Roller och andra åtkomstkontroller bör användas för att begränsa vilka operationer som olika användare får utföra. Kontrollen bör genomföras på serversidan eftersom klienten inte kan betraktas som en betrodd part.

B.2.3 Felkonfiguration av molnresurser

Varför problemet uppstår

Molntjänster innehåller många konfigurationsmöjligheter för exempelvis nätverk, identiteter, brandväggar och åtkomst. Ett vanligt säkerhetsproblem är att resurser konfigureras med större åtkomst än vad som krävs, exempelvis genom onödigt publik nätverksåtkomst eller för generösa behörigheter.

Konsekvenser

En felkonfigurerad resurs kan bli åtkomlig från internet eller från identiteter som inte borde ha åtkomst. Det kan öka risken för dataläckage, obehöriga ändringar eller att en angripare kan använda resursen som ett steg vidare in i miljön.

Förebyggande

Molnresurser bör konfigureras enligt Least Privilege och Secure by Default. Nätverksåtkomst bör begränsas till det som faktiskt krävs och behörigheter bör ges på så låg nivå som möjligt. Konfigurationer bör dessutom regelbundet granskas och övervakas för att upptäcka onödigt öppen åtkomst.

B.3 OWASP Top 10

OWASP Top 10 är en sammanställning av de vanligaste och mest betydelsefulla säkerhetsriskerna inom utveckling av webbapplikationer. Listan publiceras av Open Worldwide Application Security Project (OWASP) och är avsedd att fungera som ett stöd för utvecklare, arkitekter och säkerhetsteam vid identifiering och hantering av säkerhetsrisker.

OWASP Top 10 är inte en lista över specifika attacker som måste förekomma i en applikation, utan ett sätt att uppmärksamma återkommande kategorier av säkerhetsproblem. Exempel på områden som behandlas är brister i åtkomstkontroll, kryptografiska problem, injektionsattacker och säkerhetsfel i applikationens arkitektur.

Listan är relevant för utvecklare av molnbaserade applikationer eftersom molnapplikationer ofta exponeras över internet och kan vara beroende av flera externa tjänster och komponenter. Ett säkerhetsproblem i själva applikationen kan därför få konsekvenser även för de molnresurser som applikationen har åtkomst till.

I den aktuella lösningen är exempelvis **Broken Access Control** relevant eftersom API:t innehåller olika roller med olika behörigheter. Om auktoriseringen implementeras felaktigt skulle en användare kunna få tillgång till funktioner som rollen inte ska ha.

OWASP Top 10 kan därför användas som ett praktiskt stöd under utveckling och säkerhetsgranskning. Genom att jämföra applikationen mot de riskområden som OWASP identifierar kan utvecklaren upptäcka potentiella problem och vidta åtgärder innan applikationen tas i produktion.

Bilaga C – Angriparens arbetssätt och verktyg

C.1 Angriparens arbetssätt

Ett angrepp mot en molnbaserad applikation sker ofta i flera steg. Exakt tillvägagångssätt varierar beroende på vilken typ av applikation och vilka skydd som finns, men ett angrepp kan exempelvis börja med informationsinsamling och därefter övergå till försök att utnyttja identifierade svagheter.

Informationsinsamling

Det första steget kan vara att samla information om den organisation, applikation eller infrastruktur som angriparen vill angripa. Det kan exempelvis handla om vilka tjänster som är publikt åtkomliga, vilka tekniker som används och vilka API-endpoints som finns.

Syftet är att skapa en bild av attackytan och identifiera möjliga angreppsvägar.

Försök att utnyttja sårbarheter

När angriparen har identifierat potentiella svagheter kan olika försök göras för att utnyttja dem. Det kan exempelvis handla om brister i autentisering, auktorisering, indatahantering eller konfiguration.

En angripare kan även försöka använda kända sårbarheter i föråldrade bibliotek eller andra komponenter som applikationen är beroende av.

Åtkomst till resurser

Om en sårbarhet kan utnyttjas kan angriparen försöka få åtkomst till information eller funktioner som egentligen inte ska vara tillgängliga. I en molnmiljö kan detta även innebära att angriparen försöker utnyttja applikationens identitet för att komma åt andra Azure-resurser.

Hur stor påverkan detta får beror bland annat på vilka behörigheter den komprometterade identiteten har.

Eskalering av behörigheter

Efter att angriparen fått någon form av åtkomst kan nästa mål vara att få högre behörigheter. Detta kan exempelvis ske genom att utnyttja ytterligare sårbarheter eller felaktiga behörighetstilldelningar.

Om en identitet har större behörigheter än vad som krävs kan en sådan attack få betydligt större konsekvenser. Least Privilege är därför en viktig säkerhetsprincip även vid ett redan påbörjat intrång.

C.2 Vanliga verktyg och metoder

Sårbarhetsskanning

Automatiserade sårbarhetsskannrar kan användas för att identifiera kända säkerhetsproblem i exempelvis webbapplikationer, servrar och programvaruberoenden. Verktygen kan upptäcka exempelvis föråldrade komponenter, kända sårbarheter och vissa typer av felkonfigurationer.

Utvecklare kan minska risken genom att hålla beroenden uppdaterade, regelbundet genomföra säkerhetsskanningar och åtgärda identifierade sårbarheter.

Credential attacks

Angripare kan försöka få tillgång till användarkonton genom exempelvis återanvända eller läckta lösenord, lösenordsgissning eller andra metoder som riktas mot autentiseringen.

Risken kan minskas genom stark autentisering, multifaktorautentisering, säkra identitetslösningar och genom att aldrig lagra lösenord eller andra autentiseringsuppgifter direkt i källkoden.

I molnmiljöer bör även applikationers egna credentials undvikas när Managed Identity eller motsvarande identitetsmekanismer kan användas.

C.3 Betydelsen av flera säkerhetslager

Angriparens arbetssätt visar varför en molnapplikation behöver skyddas på flera nivåer. Ett angrepp som lyckas passera ett säkerhetslager behöver inte automatiskt innebära att angriparen får full åtkomst till systemet.

Autentisering, rollbaserad auktorisering, Least Privilege, nätverksbegränsningar, säker hantering av secrets, uppdaterade beroenden samt loggning och övervakning kan tillsammans begränsa angriparens möjligheter och minska konsekvenserna av ett intrång.

Bilaga D – Säkerhetsterminologi

D.1 Authentication

Authentication innebär att verifiera identiteten hos en användare, applikation eller annan entitet.

Exempel: En användare loggar in med Microsoft Entra ID. Entra ID verifierar användarens identitet och utfärdar därefter en access token.

D.2 Authorization

Authorization innebär att avgöra vad en autentiserad identitet har rätt att göra och vilka resurser den får använda.

Exempel: API:t kontrollerar användarens roll och tillåter endast en PropertyManager att uppdatera eller radera en felanmälan.

D.3 Least Privilege

Least Privilege innebär att en användare, applikation eller tjänst endast ska ha de behörigheter som krävs för att utföra sin uppgift.

Exempel: En Managed Identity för App Service får endast den åtkomst till Key Vault som applikationen behöver, istället för administratörsbehörighet.

D.4 Defense in Depth

Defense in Depth innebär att flera säkerhetslager används tillsammans så att ett enskilt säkerhetsfel inte automatiskt leder till ett fullständigt intrång.

Exempel: Autentisering, rollbaserad auktorisering, nätverksskydd, Key Vault, Managed Identity och kryptering kan tillsammans ge flera lager av skydd.

D.5 Zero Trust

Zero Trust är en säkerhetsprincip som innebär att åtkomst inte ska beviljas automatiskt baserat på exempelvis att en användare befinner sig i ett internt nätverk. Varje åtkomst ska istället verifieras och endast tillåtas när den är motiverad.

En central tanke är "**never trust, always verify**".

Exempel: En användare måste autentiseras och ha rätt roll för att få använda en viss API-endpoint, även om användaren redan har tillgång till organisationens miljö.

D.6 RBAC

RBAC (Role-Based Access Control) innebär att behörigheter kopplas till roller istället för att varje behörighet behöver tilldelas individuellt till varje användare.

Exempel: I lösningen används rollerna Resident och PropertyManager för att styra vilka API-operationer användarna får utföra. Azure använder också RBAC för behörigheter till Azure-resurser.

D.7 Secret

Ett **Secret** är känslig information som används för autentisering eller åtkomst till en tjänst eller resurs och därför måste skyddas.

Exempel: API-nycklar, lösenord och Client Secrets är exempel på secrets. I lösningen lagras SMSApiKey i Azure Key Vault.

D.8 Encryption at Rest

Encryption at Rest innebär att data krypteras när den är lagrad, exempelvis i en databas, lagringstjänst eller annan persistent lagring.

Syftet är att skydda information om den lagrade datan skulle bli åtkomlig för en obehörig part.

Exempel: Data som lagras i Azure SQL skyddas genom kryptering av data i vila.

D.9 Encryption in Transit

Encryption in Transit innebär att information krypteras när den överförs mellan exempelvis en klient och ett API eller mellan olika tjänster.

Exempel: HTTPS används för att skydda kommunikationen mellan en klient och API:t.

D.10 Managed Identity

Managed Identity är en identitet som hanteras av Azure och som kan användas av en Azure-resurs för att autentisera sig mot andra Azure-tjänster utan att utvecklaren behöver lagra credentials i applikationen.

Exempel: App Service använder en Managed Identity för att autentisera sig mot Azure Key Vault och läsa den API-nyckel som SMSNotifier behöver.

D.11 Secure by Design

Secure by Design innebär att säkerhet ska vara en del av systemets arkitektur och utveckling från början, istället för att läggas till först efter att systemet har byggts.

Exempel: I lösningen har autentisering, rollbaserad auktorisering och säker hantering av secrets byggts in som en del av API:ets design.

D.12 Secure by Default

Secure by Default innebär att ett system ska ha säkra standardinställningar och att användaren eller administratören inte ska behöva göra extra säkerhetskfiguration för att grundläggande skydd ska vara aktiverat.

Exempel: En applikation bör som utgångspunkt inte ge användare eller tjänster mer åtkomst än vad som krävs. Funktioner och resurser bör inte vara publikt åtkomliga om det inte finns ett uttryckligt behov.